

The JavaScript language

JS

Day5

Error handling

Error handling, "try...catch"

No matter how great we are at programming, sometimes our scripts have errors. They may occur because of our mistakes, an unexpected user input, an erroneous server response, and for a thousand other reasons.

Usually, a script “dies” (immediately stops) in case of an error, printing it to console.

But there’s a syntax construct `try...catch` that allows us to “catch” errors so the script can, instead of dying, do something more reasonable.

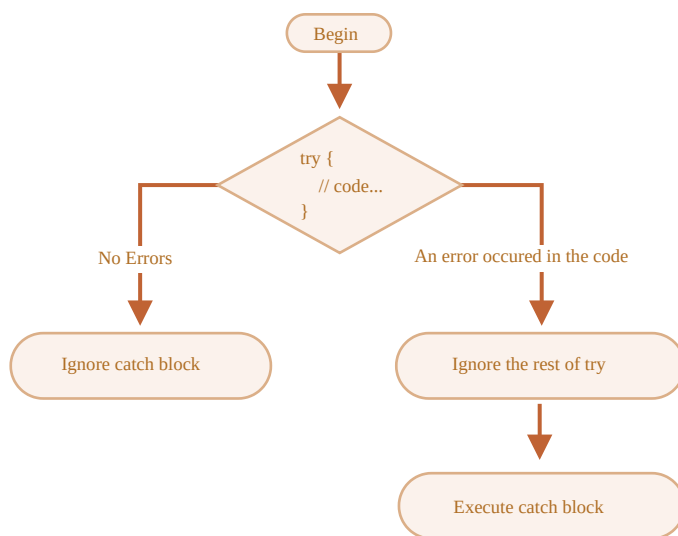
The “try...catch” syntax

The `try...catch` construct has two main blocks: `try`, and then `catch`:

```
try {  
    // code...  
} catch (err) {  
    // error handling  
}
```

It works like this:

1. First, the code in `try {...}` is executed.
2. If there were no errors, then `catch (err)` is ignored: the execution reaches the end of `try` and goes on, skipping `catch`.
3. If an error occurs, then the `try` execution is stopped, and control flows to the beginning of `catch (err)`. The `err` variable (we can use any name for it) will contain an error object with details about what happened.



So, an error inside the `try {...}` block does not kill the script – we have a chance to handle it in `catch`.

Let's look at some examples.

- An errorless example: shows `alert (1)` and `(2)`:

```
try {  
    alert('Start of try runs'); // (1) <--  
    // ...no errors here  
    alert('End of try runs'); // (2) <--  
} catch (err) {  
    alert('Catch is ignored, because there are no errors'); // (3)  
}
```

- An example with an error: shows `(1)` and `(3)`:

```
try {  
    alert('Start of try runs'); // (1) <--
```

```

lala; // error, variable is not defined!

alert('End of try (never reached)'); // (2)

} catch (err) {

    alert('Error has occurred!'); // (3) <--

}

```

⚠ `try...catch` only works for runtime errors

For `try...catch` to work, the code must be runnable. In other words, it should be valid JavaScript.

It won't work if the code is syntactically wrong, for instance it has unmatched curly braces:

```

try {
    {}{}{}{}{}{}{}{}
} catch (err) {
    alert("The engine can't understand this code, it's invalid");
}

```

The JavaScript engine first reads the code, and then runs it. The errors that occur on the reading phase are called “parse-time” errors and are unrecoverable (from inside that code). That’s because the engine can’t understand the code.

So, `try...catch` can only handle errors that occur in valid code. Such errors are called “runtime errors” or, sometimes, “exceptions”.

⚠ `try...catch` works synchronously

If an exception happens in “scheduled” code, like in `setTimeout`, then `try...catch` won't catch it:

```

try {
    setTimeout(function() {
        noSuchVariable; // script will die here
    }, 1000);
} catch (err) {
    alert( "won't work" );
}

```

That's because the function itself is executed later, when the engine has already left the `try...catch` construct.

To catch an exception inside a scheduled function, `try...catch` must be inside that function:

```

setTimeout(function() {
    try {
        noSuchVariable; // try...catch handles the error!
    } catch {
        alert( "error is caught here!" );
    }
}, 1000);

```

Error object

When an error occurs, JavaScript generates an object containing the details about it. The object is then passed as an argument to `catch`:

```

try {
    // ...
} catch (err) { // <-- the "error object", could use another word instead of err
    // ...
}

```

For all built-in errors, the error object has two main properties:

name

Error name. For instance, for an undefined variable that's `"ReferenceError"`.

message

Textual message about error details.

There are other non-standard properties available in most environments. One of most widely used and supported is:

stack

Current call stack: a string with information about the sequence of nested calls that led to the error. Used for debugging purposes.

For instance:

```
try {
  lalala; // error, variable is not defined!
} catch (err) {
  alert(err.name); // ReferenceError
  alert(err.message); // lalala is not defined
  alert(err.stack); // ReferenceError: lalala is not defined at (...call stack)

  // Can also show an error as a whole
  // The error is converted to string as "name: message"
  alert(err); // ReferenceError: lalala is not defined
}
```

Optional “catch” binding

A recent addition

This is a recent addition to the language. Old browsers may need [polyfills](#).

If we don't need error details, `catch` may omit it:

```
try {
  // ...
} catch { // <-- without (err)
  // ...
}
```

Using “try...catch”

Let's explore a real-life use case of `try...catch`.

As we already know, JavaScript supports the `JSON.parse(str)` [↗](#) method to read JSON-encoded values.

Usually it's used to decode data received over the network, from the server or another source.

We receive it and call `JSON.parse` like this:

```
let json = '{"name":"John", "age": 30}'; // data from the server

let user = JSON.parse(json); // convert the text representation to JS object

// now user is an object with properties from the string
alert( user.name ); // John
alert( user.age ); // 30
```

You can find more detailed information about JSON in the [JSON methods](#), [toJSON](#) chapter.

If `json` is malformed, `JSON.parse` generates an error, so the script “dies”.

Should we be satisfied with that? Of course not!

This way, if something's wrong with the data, the visitor will never know that (unless they open the developer console). And people really don't like when something “just dies” without any error message.

Let's use `try...catch` to handle the error:

```
let json = "{ bad json }";

try {

  let user = JSON.parse(json); // <-- when an error occurs...
  alert( user.name ); // doesn't work

} catch (err) {
  // ...the execution jumps here
  alert( "Our apologies, the data has errors, we'll try to request it one more time." );
  alert( err.name );
  alert( err.message );
}
```

Here we use the `catch` block only to show the message, but we can do much more: send a new network request, suggest an alternative to the visitor, send information about the error to a logging facility, All much better than just dying.

Throwing our own errors

What if `json` is syntactically correct, but doesn't have a required `name` property?

Like this:

```
let json = '{ "age": 30 }'; // incomplete data

try {

  let user = JSON.parse(json); // <-- no errors
  alert( user.name ); // no name!

} catch (err) {
  alert( "doesn't execute" );
}
```

Here `JSON.parse` runs normally, but the absence of `name` is actually an error for us.

To unify error handling, we'll use the `throw` operator.

“Throw” operator

The `throw` operator generates an error.

The syntax is:

```
throw <error object>
```

Technically, we can use anything as an error object. That may be even a primitive, like a number or a string, but it's better to use objects, preferably with `name` and `message` properties (to stay somewhat compatible with built-in errors).

JavaScript has many built-in constructors for standard errors: `Error`, `SyntaxError`, `ReferenceError`, `TypeError` and others. We can use them to create error objects as well.

Their syntax is:

```
let error = new Error(message);
// or
let error = new SyntaxError(message);
let error = new ReferenceError(message);
// ...
```

For built-in errors (not for any objects, just for errors), the `name` property is exactly the name of the constructor. And `message` is taken from the argument.

For instance:

```
let error = new Error("Things happen o_0");

alert(error.name); // Error
alert(error.message); // Things happen o_0
```

Let's see what kind of error `JSON.parse` generates:

```
try {
    JSON.parse("{ bad json o_0 }");
} catch (err) {
    alert(err.name); // SyntaxError
    alert(err.message); // Unexpected token b in JSON at position 2
}
```

As we can see, that's a `SyntaxError`.

And in our case, the absence of `name` is an error, as users must have a `name`.

So let's throw it:

```
let json = '{ "age": 30 }'; // incomplete data

try {

    let user = JSON.parse(json); // <-- no errors

    if (!user.name) {
        throw new SyntaxError("Incomplete data: no name"); // (*)
    }

    alert( user.name );

} catch (err) {
    alert( "JSON Error: " + err.message ); // JSON Error: Incomplete data: no name
}
```

In the line `(*)`, the `throw` operator generates a `SyntaxError` with the given `message`, the same way as JavaScript would generate it itself. The execution of `try` immediately stops and the control flow jumps into `catch`.

Now `catch` became a single place for all error handling: both for `JSON.parse` and other cases.

Rethrowing

In the example above we use `try...catch` to handle incorrect data. But is it possible that *another unexpected error* occurs within the `try {...}` block? Like a programming error (variable is not defined) or something else, not just this “incorrect data” thing.

For example:

```
let json = '{ "age": 30 }'; // incomplete data

try {
    user = JSON.parse(json); // <-- forgot to put "let" before user

    // ...
} catch (err) {
    alert("JSON Error: " + err); // JSON Error: ReferenceError: user is not defined
    // (no JSON Error actually)
}
```

Of course, everything's possible! Programmers do make mistakes. Even in open-source utilities used by millions for decades – suddenly a bug may be discovered that leads to terrible hacks.

In our case, `try...catch` is placed to catch “incorrect data” errors. But by its nature, `catch` gets *all* errors from `try`. Here it gets an unexpected error, but still shows the same “JSON Error” message. That's wrong and also makes the code more difficult to debug.

To avoid such problems, we can employ the “rethrowing” technique. The rule is simple:

Catch should only process errors that it knows and “rethrow” all others.

The “rethrowing” technique can be explained in more detail as:

1. Catch gets all errors.
2. In the `catch (err) { ... }` block we analyze the error object `err`.
3. If we don't know how to handle it, we do `throw err`.

Usually, we can check the error type using the `instanceof` operator:

```
try {
  user = { /*...*/ };
} catch (err) {
  if (err instanceof ReferenceError) {
    alert('ReferenceError'); // "ReferenceError" for accessing an undefined variable
  }
}
```

We can also get the error class name from `err.name` property. All native errors have it. Another option is to read `err.constructor.name`.

In the code below, we use rethrowing so that `catch` only handles `SyntaxError`:

```
let json = '{ "age": 30 }'; // incomplete data
try {

  let user = JSON.parse(json);

  if (!user.name) {
    throw new SyntaxError("Incomplete data: no name");
  }

  blabla(); // unexpected error

  alert( user.name );

} catch (err) {

  if (err instanceof SyntaxError) {
    alert( "JSON Error: " + err.message );
  } else {
    throw err; // rethrow (*)
  }

}
```

The error throwing on line `(*)` from inside `catch` block “falls out” of `try...catch` and can be either caught by an outer `try...catch` construct (if it exists), or it kills the script.

So the `catch` block actually handles only errors that it knows how to deal with and “skips” all others.

The example below demonstrates how such errors can be caught by one more level of `try...catch`:

```
function readData() {
  let json = '{ "age": 30 }';

  try {
    // ...
    blabla(); // error!
  } catch (err) {
    // ...
    if (!(err instanceof SyntaxError)) {
      throw err; // rethrow (don't know how to deal with it)
    }
  }
}

try {
  readData();
} catch (err) {
```

```
    alert( "External catch got: " + err ); // caught it!
}
```

Here `readData` only knows how to handle `SyntaxError`, while the outer `try...catch` knows how to handle everything.

try...catch...finally

Wait, that's not all.

The `try...catch` construct may have one more code clause: `finally`.

If it exists, it runs in all cases:

- after `try`, if there were no errors,
- after `catch`, if there were errors.

The extended syntax looks like this:

```
try {
    ... try to execute the code ...
} catch (err) {
    ... handle errors ...
} finally {
    ... execute always ...
}
```

Try running this code:

```
try {
    alert( 'try' );
    if (confirm('Make an error?')) BAD_CODE();
} catch (err) {
    alert( 'catch' );
} finally {
    alert( 'finally' );
}
```

The code has two ways of execution:

1. If you answer “Yes” to “Make an error?”, then `try -> catch -> finally`.
2. If you say “No”, then `try -> finally`.

The `finally` clause is often used when we start doing something and want to finalize it in any case of outcome.

For instance, we want to measure the time that a Fibonacci numbers function `fib(n)` takes. Naturally, we can start measuring before it runs and finish afterwards. But what if there's an error during the function call? In particular, the implementation of `fib(n)` in the code below returns an error for negative or non-integer numbers.

The `finally` clause is a great place to finish the measurements no matter what.

Here `finally` guarantees that the time will be measured correctly in both situations – in case of a successful execution of `fib` and in case of an error in it:

```
let num = +prompt("Enter a positive integer number?", 35)

let diff, result;

function fib(n) {
    if (n < 0 || Math.trunc(n) !== n) {
        throw new Error("Must not be negative, and also an integer.");
    }
    return n <= 1 ? n : fib(n - 1) + fib(n - 2);
}

let start = Date.now();

try {
```

```

    result = fib(num);
  } catch (err) {
    result = 0;
  } finally {
    diff = Date.now() - start;
  }

  alert(result || "error occurred");

  alert( `execution took ${diff}ms` );

```

You can check by running the code with entering `35` into `prompt` – it executes normally, `finally` after `try`. And then enter `-1` – there will be an immediate error, and the execution will take `0ms`. Both measurements are done correctly.

In other words, the function may finish with `return` or `throw`, that doesn't matter. The `finally` clause executes in both cases.

Variables are local inside `try...catch...finally`

Please note that `result` and `diff` variables in the code above are declared *before* `try...catch`.

Otherwise, if we declared `let` in `try` block, it would only be visible inside of it.

`finally` and `return`

The `finally` clause works for *any* exit from `try...catch`. That includes an explicit `return`.

In the example below, there's a `return` in `try`. In this case, `finally` is executed just before the control returns to the outer code.

```

function func() {

  try {
    return 1;

  } catch (err) {
    /* ... */
  } finally {
    alert( 'finally' );
  }
}

alert( func() ); // first works alert from finally, and then this one

```

`try...finally`

The `try...finally` construct, without `catch` clause, is also useful. We apply it when we don't want to handle errors here (let them fall through), but want to be sure that processes that we started are finalized.

```

function func() {
  // start doing something that needs completion (like measurements)
  try {
    // ...
  } finally {
    // complete that thing even if all dies
  }
}

```

In the code above, an error inside `try` always falls out, because there's no `catch`. But `finally` works before the execution flow leaves the function.

Global catch

Environment-specific

The information from this section is not a part of the core JavaScript.

Let's imagine we've got a fatal error outside of `try...catch`, and the script died. Like a programming error or some other terrible thing.

Is there a way to react on such occurrences? We may want to log the error, show something to the user (normally they don't see error messages), etc.

There is none in the specification, but environments usually provide it, because it's really useful. For instance, Node.js has `process.on("uncaughtException")` [↗](#) for that. And in the browser we can assign a function to the special `window.onerror` [↗](#) property, that will run in case of an uncaught error.

The syntax:

```
window.onerror = function(message, url, line, col, error) {  
  // ...  
};
```

`message`

Error message.

`url`

URL of the script where error happened.

`line, col`

Line and column numbers where error happened.

`error`

Error object.

For instance:

```
<script>  
  window.onerror = function(message, url, line, col, error) {  
    alert(`${message}\n At ${line}:${col} of ${url}`);  
  };  
  
  function readData() {  
    badFunc(); // Whoops, something went wrong!  
  }  
  
  readData();  
</script>
```

The role of the global handler `window.onerror` is usually not to recover the script execution – that's probably impossible in case of programming errors, but to send the error message to developers.

There are also web-services that provide error-logging for such cases, like <https://muscula.com> [↗](#) or <https://www.sentry.io> [↗](#).

They work like this:

1. We register at the service and get a piece of JS (or a script URL) from them to insert on pages.
2. That JS script sets a custom `window.onerror` function.
3. When an error occurs, it sends a network request about it to the service.
4. We can log in to the service web interface and see errors.

Summary

The `try...catch` construct allows to handle runtime errors. It literally allows to “try” running the code and “catch” errors that may occur in it.

The syntax is:

```
try {  
  // run this code  
} catch (err) {  
  // if an error happened, then jump here
```

```
// err is the error object
} finally {
  // do in any case after try/catch
}
```

There may be no `catch` section or no `finally`, so shorter constructs `try...catch` and `try...finally` are also valid.

Error objects have following properties:

- `message` – the human-readable error message.
- `name` – the string with error name (error constructor name).
- `stack` (non-standard, but well-supported) – the stack at the moment of error creation.

If an error object is not needed, we can omit it by using `catch {}` instead of `catch (err) {}`.

We can also generate our own errors using the `throw` operator. Technically, the argument of `throw` can be anything, but usually it's an error object inheriting from the built-in `Error` class. More on extending errors in the next chapter.

Rethrowing is a very important pattern of error handling: a `catch` block usually expects and knows how to handle the particular error type, so it should rethrow errors it doesn't know.

Even if we don't have `try...catch`, most environments allow us to setup a “global” error handler to catch errors that “fall out”. In-browser, that's `window.onerror`.

✓ Tasks

Finally or just the code?

importance: 5

Compare the two code fragments.

1.

The first one uses `finally` to execute the code after `try...catch`:

```
try {
  work work
} catch (err) {
  handle errors
} finally {
  cleanup the working space
}
```

2.

The second fragment puts the cleaning right after `try...catch`:

```
try {
  work work
} catch (err) {
  handle errors
}

cleanup the working space
```

We definitely need the cleanup after the work, doesn't matter if there was an error or not.

Is there an advantage here in using `finally` or both code fragments are equal? If there is such an advantage, then give an example when it matters.

[To solution](#)

```

    }
}

function readUser(json) {
    let user;

    try {
        user = JSON.parse(json);
    } catch (err) {
        if (err instanceof SyntaxError) {
            throw new ReadError("Syntax Error", err);
        } else {
            throw err;
        }
    }

    try {
        validateUser(user);
    } catch (err) {
        if (err instanceof ValidationError) {
            throw new ReadError("Validation Error", err);
        } else {
            throw err;
        }
    }
}

try {
    readUser('{bad json}');
} catch (e) {
    if (e instanceof ReadError) {
        alert(e);
        // Original error: SyntaxError: Unexpected token b in JSON at position 1
        alert("Original error: " + e.cause);
    } else {
        throw e;
    }
}

```

In the code above, `readUser` works exactly as described – catches syntax and validation errors and throws `ReadError` errors instead (unknown errors are rethrown as usual).

So the outer code checks `instanceof ReadError` and that's it. No need to list all possible error types.

The approach is called “wrapping exceptions”, because we take “low level” exceptions and “wrap” them into `ReadError` that is more abstract. It is widely used in object-oriented programming.

Summary

- We can inherit from `Error` and other built-in error classes normally. We just need to take care of the `name` property and don't forget to call `super`.
- We can use `instanceof` to check for particular errors. It also works with inheritance. But sometimes we have an error object coming from a 3rd-party library and there's no easy way to get its class. Then `name` property can be used for such checks.
- Wrapping exceptions is a widespread technique: a function handles low-level exceptions and creates higher-level errors instead of various low-level ones. Low-level exceptions sometimes become properties of that object like `err.cause` in the examples above, but that's not strictly required.

✓ Tasks

Inherit from `SyntaxError`

importance: 5

Create a class `FormatError` that inherits from the built-in `SyntaxError` class.

It should support `message`, `name` and `stack` properties.

Usage example:

```
let err = new FormatError("formatting error");

alert( err.message ); // formatting error
alert( err.name ); // FormatError
alert( err.stack ); // stack

alert( err instanceof FormatError ); // true
alert( err instanceof SyntaxError ); // true (because inherits from SyntaxError)
```

To solution

Promises, async/await

Introduction: callbacks

We use browser methods in examples here

To demonstrate the use of callbacks, promises and other abstract concepts, we'll be using some browser methods: specifically, loading scripts and performing simple document manipulations.

If you're not familiar with these methods, and their usage in the examples is confusing, you may want to read a few chapters from the [next part](#) of the tutorial.

Although, we'll try to make things clear anyway. There won't be anything really complex browser-wise.

Many functions are provided by JavaScript host environments that allow you to schedule *asynchronous* actions. In other words, actions that we initiate now, but they finish later.

For instance, one such function is the `setTimeout` function.

There are other real-world examples of asynchronous actions, e.g. loading scripts and modules (we'll cover them in later chapters).

Take a look at the function `loadScript(src)`, that loads a script with the given `src`:

```
function loadScript(src) {
  // creates a <script> tag and append it to the page
  // this causes the script with given src to start loading and run when complete
  let script = document.createElement('script');
  script.src = src;
  document.head.append(script);
}
```

It inserts into the document a new, dynamically created, tag `<script src="...">` with the given `src`. The browser automatically starts loading it and executes when complete.

We can use this function like this:

```
// load and execute the script at the given path
loadScript('/my/script.js');
```

The script is executed “asynchronously”, as it starts loading now, but runs later, when the function has already finished.

If there's any code below `loadScript(...)`, it doesn't wait until the script loading finishes.

```
loadScript('/my/script.js');
// the code below loadScript
// doesn't wait for the script loading to finish
// ...
```

Let's say we need to use the new script as soon as it loads. It declares new functions, and we want to run them.

But if we do that immediately after the `loadScript(...)` call, that wouldn't work:

```
loadScript('/my/script.js'); // the script has "function newFunction() {...}"  
  
newFunction(); // no such function!
```

Naturally, the browser probably didn't have time to load the script. As of now, the `loadScript` function doesn't provide a way to track the load completion. The script loads and eventually runs, that's all. But we'd like to know when it happens, to use new functions and variables from that script.

Let's add a `callback` function as a second argument to `loadScript` that should execute when the script loads:

```
function loadScript(src, callback) {  
  let script = document.createElement('script');  
  script.src = src;  
  
  script.onload = () => callback(script);  
  
  document.head.append(script);  
}
```

The `onload` event is described in the article [Resource loading: onload and onerror](#), it basically executes a function after the script is loaded and executed.

Now if we want to call new functions from the script, we should write that in the callback:

```
loadScript('/my/script.js', function() {  
  // the callback runs after the script is loaded  
  newFunction(); // so now it works  
  ...  
});
```

That's the idea: the second argument is a function (usually anonymous) that runs when the action is completed.

Here's a runnable example with a real script:

```
function loadScript(src, callback) {  
  let script = document.createElement('script');  
  script.src = src;  
  script.onload = () => callback(script);  
  document.head.append(script);  
}  
  
loadScript('https://cdnjs.cloudflare.com/ajax/libs/lodash.js/3.2.0/lodash.js', script => {  
  alert(`Cool, the script ${script.src} is loaded`);  
  alert( _ ); // _ is a function declared in the loaded script  
});
```

That's called a "callback-based" style of asynchronous programming. A function that does something asynchronously should provide a `callback` argument where we put the function to run after it's complete.

Here we did it in `loadScript`, but of course it's a general approach.

Callback in callback

How can we load two scripts sequentially: the first one, and then the second one after it?

The natural solution would be to put the second `loadScript` call inside the callback, like this:

```
loadScript('/my/script.js', function(script) {  
  
  alert(`Cool, the ${script.src} is loaded, let's load one more`);  
  
  loadScript('/my/script2.js', function(script) {  
    alert(`Cool, the second script is loaded`);  
  });  
});
```

After the outer `loadScript` is complete, the callback initiates the inner one.

What if we want one more script...?

```
loadScript('/my/script.js', function(script) {  
  loadScript('/my/script2.js', function(script) {  
    loadScript('/my/script3.js', function(script) {  
      // ...continue after all scripts are loaded  
    });  
  });  
});
```

So, every new action is inside a callback. That's fine for few actions, but not good for many, so we'll see other variants soon.

Handling errors

In the above examples we didn't consider errors. What if the script loading fails? Our callback should be able to react on that.

Here's an improved version of `loadScript` that tracks loading errors:

```
function loadScript(src, callback) {  
  let script = document.createElement('script');  
  script.src = src;  
  
  script.onload = () => callback(null, script);  
  script.onerror = () => callback(new Error(`Script load error for ${src}`));  
  
  document.head.append(script);  
}
```

It calls `callback(null, script)` for successful load and `callback(error)` otherwise.

The usage:

```
loadScript('/my/script.js', function(error, script) {  
  if (error) {  
    // handle error  
  } else {  
    // script loaded successfully  
  }  
});
```

Once again, the recipe that we used for `loadScript` is actually quite common. It's called the "error-first callback" style.

The convention is:

1. The first argument of the `callback` is reserved for an error if it occurs. Then `callback(err)` is called.
2. The second argument (and the next ones if needed) are for the successful result. Then `callback(null, result1, result2...)` is called.

So the single `callback` function is used both for reporting errors and passing back results.

Pyramid of Doom

At first glance, it looks like a viable approach to asynchronous coding. And indeed it is. For one or maybe two nested calls it looks fine.

But for multiple asynchronous actions that follow one after another, we'll have code like this:

```
loadScript('1.js', function(error, script) {  
  
  if (error) {  
    handleError(error);  
  }  
});
```

```

} else {
  // ...
  loadScript('2.js', function(error, script) {
    if (error) {
      handleError(error);
    } else {
      // ...
      loadScript('3.js', function(error, script) {
        if (error) {
          handleError(error);
        } else {
          // ...continue after all scripts are loaded (*)
        }
      });
    }
  });
}
});

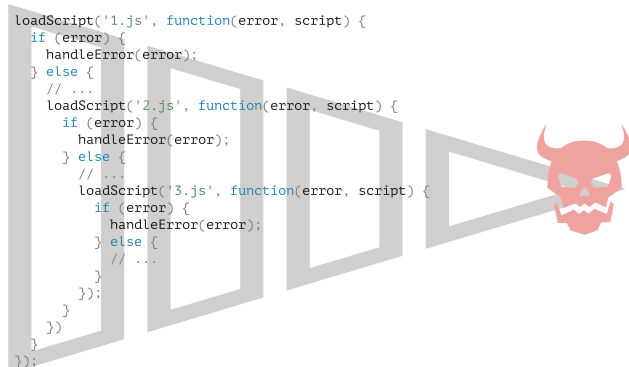
```

In the code above:

1. We load `1.js`, then if there's no error...
2. We load `2.js`, then if there's no error...
3. We load `3.js`, then if there's no error – do something else `(*)`.

As calls become more nested, the code becomes deeper and increasingly more difficult to manage, especially if we have real code instead of `...` that may include more loops, conditional statements and so on.

That's sometimes called “callback hell” or “pyramid of doom.”



```

loadScript('1.js', function(error, script) {
  if (error) {
    handleError(error);
  } else {
    // ...
    loadScript('2.js', function(error, script) {
      if (error) {
        handleError(error);
      } else {
        // ...
        loadScript('3.js', function(error, script) {
          if (error) {
            handleError(error);
          } else {
            // ...
          }
        });
      }
    });
  }
});

```

The “pyramid” of nested calls grows to the right with every asynchronous action. Soon it spirals out of control.

So this way of coding isn't very good.

We can try to alleviate the problem by making every action a standalone function, like this:

```

loadScript('1.js', step1);

function step1(error, script) {
  if (error) {
    handleError(error);
  } else {
    // ...
    loadScript('2.js', step2);
  }
}

function step2(error, script) {
  if (error) {
    handleError(error);
  } else {
    // ...
    loadScript('3.js', step3);
  }
}

function step3(error, script) {
  if (error) {

```

```

    handleError(error);
  } else {
    // ...continue after all scripts are loaded (*)
  }
}

```

See? It does the same thing, and there's no deep nesting now because we made every action a separate top-level function.

It works, but the code looks like a torn apart spreadsheet. It's difficult to read, and you probably noticed that one needs to eye-jump between pieces while reading it. That's inconvenient, especially if the reader is not familiar with the code and doesn't know where to eye-jump.

Also, the functions named `step*` are all of single use, they are created only to avoid the “pyramid of doom.” No one is going to reuse them outside of the action chain. So there's a bit of namespace cluttering here.

We'd like to have something better.

Luckily, there are other ways to avoid such pyramids. One of the best ways is to use “promises”, described in the next chapter.

Promise

Imagine that you're a top singer, and fans ask day and night for your upcoming song.

To get some relief, you promise to send it to them when it's published. You give your fans a list. They can fill in their email addresses, so that when the song becomes available, all subscribed parties instantly receive it. And even if something goes very wrong, say, a fire in the studio, so that you can't publish the song, they will still be notified.

Everyone is happy: you, because the people don't crowd you anymore, and fans, because they won't miss the song.

This is a real-life analogy for things we often have in programming:

1. A “producing code” that does something and takes time. For instance, some code that loads the data over a network. That's a “singer”.
2. A “consuming code” that wants the result of the “producing code” once it's ready. Many functions may need that result. These are the “fans”.
3. A *promise* is a special JavaScript object that links the “producing code” and the “consuming code” together. In terms of our analogy: this is the “subscription list”. The “producing code” takes whatever time it needs to produce the promised result, and the “promise” makes that result available to all of the subscribed code when it's ready.

The analogy isn't terribly accurate, because JavaScript promises are more complex than a simple subscription list: they have additional features and limitations. But it's fine to begin with.

The constructor syntax for a promise object is:

```

let promise = new Promise(function(resolve, reject) {
  // executor (the producing code, "singer")
});

```

The function passed to `new Promise` is called the *executor*. When `new Promise` is created, the executor runs automatically. It contains the producing code which should eventually produce the result. In terms of the analogy above: the executor is the “singer”.

Its arguments `resolve` and `reject` are callbacks provided by JavaScript itself. Our code is only inside the executor.

When the executor obtains the result, be it soon or late, doesn't matter, it should call one of these callbacks:

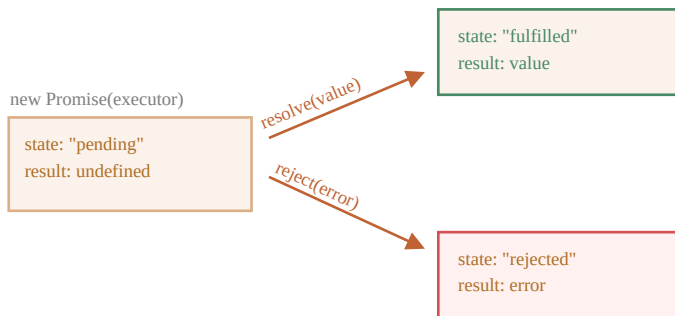
- `resolve(value)` — if the job is finished successfully, with result `value`.
- `reject(error)` — if an error has occurred, `error` is the error object.

So to summarize: the executor runs automatically and attempts to perform a job. When it is finished with the attempt, it calls `resolve` if it was successful or `reject` if there was an error.

The `promise` object returned by the `new Promise` constructor has these internal properties:

- `state` — initially `"pending"`, then changes to either `"fulfilled"` when `resolve` is called or `"rejected"` when `reject` is called.
- `result` — initially `undefined`, then changes to `value` when `resolve(value)` is called or `error` when `reject(error)` is called.

So the executor eventually moves `promise` to one of these states:



Later we'll see how "fans" can subscribe to these changes.

Here's an example of a promise constructor and a simple executor function with "producing code" that takes time (via `setTimeout`):

```
let promise = new Promise(function(resolve, reject) {  
  // the function is executed automatically when the promise is constructed  
  
  // after 1 second signal that the job is done with the result "done"  
  setTimeout(() => resolve("done"), 1000);  
});
```

We can see two things by running the code above:

1. The executor is called automatically and immediately (by `new Promise`).
2. The executor receives two arguments: `resolve` and `reject`. These functions are pre-defined by the JavaScript engine, so we don't need to create them. We should only call one of them when ready.

After one second of "processing", the executor calls `resolve("done")` to produce the result. This changes the state of the `promise` object:



That was an example of a successful job completion, a "fulfilled promise".

And now an example of the executor rejecting the promise with an error:

```
let promise = new Promise(function(resolve, reject) {  
  // after 1 second signal that the job is finished with an error  
  setTimeout(() => reject(new Error("Whoops!")), 1000);  
});
```

The call to `reject(...)` moves the promise object to "rejected" state:



To summarize, the executor should perform a job (usually something that takes time) and then call `resolve` or `reject` to change the state of the corresponding promise object.

A promise that is either resolved or rejected is called "settled", as opposed to an initially "pending" promise.

i There can be only a single result or an error

The executor should call only one `resolve` or one `reject`. Any state change is final.

All further calls of `resolve` and `reject` are ignored:

```
let promise = new Promise(function(resolve, reject) {  
  resolve("done");  
  
  reject(new Error("...")); // ignored  
  setTimeout(() => resolve("...")); // ignored  
});
```

The idea is that a job done by the executor may have only one result or an error.

Also, `resolve/reject` expect only one argument (or none) and will ignore additional arguments.

i Reject with `Error` objects

In case something goes wrong, the executor should call `reject`. That can be done with any type of argument (just like `resolve`). But it is recommended to use `Error` objects (or objects that inherit from `Error`). The reasoning for that will soon become apparent.

i Immediately calling `resolve/reject`

In practice, an executor usually does something asynchronously and calls `resolve/reject` after some time, but it doesn't have to. We also can call `resolve` or `reject` immediately, like this:

```
let promise = new Promise(function(resolve, reject) {  
  // not taking our time to do the job  
  resolve(123); // immediately give the result: 123  
});
```

For instance, this might happen when we start to do a job but then see that everything has already been completed and cached.

That's fine. We immediately have a resolved promise.

i The `state` and `result` are internal

The properties `state` and `result` of the `Promise` object are internal. We can't directly access them. We can use the methods `.then/.catch/.finally` for that. They are described below.

Consumers: `then`, `catch`

A `Promise` object serves as a link between the executor (the “producing code” or “singer”) and the consuming functions (the “fans”), which will receive the result or error. Consuming functions can be registered (subscribed) using the methods `.then` and `.catch`.

then

The most important, fundamental one is `.then`.

The syntax is:

```
promise.then(  
  function(result) { /* handle a successful result */ },  
  function(error) { /* handle an error */ }  
);
```

The first argument of `.then` is a function that runs when the promise is resolved and receives the result.

The second argument of `.then` is a function that runs when the promise is rejected and receives the error.

For instance, here's a reaction to a successfully resolved promise:

```
let promise = new Promise(function(resolve, reject) {
  setTimeout(() => resolve("done!"), 1000);
});

// resolve runs the first function in .then
promise.then(
  result => alert(result), // shows "done!" after 1 second
  error => alert(error) // doesn't run
);
```

The first function was executed.

And in the case of a rejection, the second one:

```
let promise = new Promise(function(resolve, reject) {
  setTimeout(() => reject(new Error("Whoops!")), 1000);
});

// reject runs the second function in .then
promise.then(
  result => alert(result), // doesn't run
  error => alert(error) // shows "Error: Whoops!" after 1 second
);
```

If we're interested only in successful completions, then we can provide only one function argument to `.then`:

```
let promise = new Promise(resolve => {
  setTimeout(() => resolve("done!"), 1000);
});

promise.then(alert); // shows "done!" after 1 second
```

catch

If we're interested only in errors, then we can use `null` as the first argument: `.then(null, errorHandlerFunction)`. Or we can use `.catch(errorHandlerFunction)`, which is exactly the same:

```
let promise = new Promise((resolve, reject) => {
  setTimeout(() => reject(new Error("Whoops!")), 1000);
});

// .catch(f) is the same as promise.then(null, f)
promise.catch(alert); // shows "Error: Whoops!" after 1 second
```

The call `.catch(f)` is a complete analog of `.then(null, f)`, it's just a shorthand.

Cleanup: finally

Just like there's a `finally` clause in a regular `try {...} catch {...}`, there's `finally` in promises.

The call `.finally(f)` is similar to `.then(f, f)` in the sense that `f` runs always, when the promise is settled: be it resolve or reject.

The idea of `finally` is to set up a handler for performing cleanup/finalizing after the previous operations are complete.

E.g. stopping loading indicators, closing no longer needed connections, etc.

Think of it as a party finisher. Irrespective of whether a party was good or bad, how many friends were in it, we still need (or at least should) do a cleanup after it.

The code may look like this:

```
new Promise((resolve, reject) => {
  /* do something that takes time, and then call resolve or maybe reject */
})
```

```
// runs when the promise is settled, doesn't matter successfully or not
.finally(() => stop loading indicator)
// so the loading indicator is always stopped before we go on
.then(result => show result, err => show error)
```

Please note that `finally(f)` isn't exactly an alias of `then(f, f)` though.

There are important differences:

1. A `finally` handler has no arguments. In `finally` we don't know whether the promise is successful or not. That's all right, as our task is usually to perform "general" finalizing procedures.

Please take a look at the example above: as you can see, the `finally` handler has no arguments, and the promise outcome is handled by the next handler.

2. A `finally` handler "passes through" the result or error to the next suitable handler.

For instance, here the result is passed through `finally` to `then`:

```
new Promise((resolve, reject) => {
  setTimeout(() => resolve("value"), 2000);
})
.finally(() => alert("Promise ready")) // triggers first
.then(result => alert(result)); // <-- .then shows "value"
```

As you can see, the `value` returned by the first promise is passed through `finally` to the next `then`.

That's very convenient, because `finally` is not meant to process a promise result. As said, it's a place to do generic cleanup, no matter what the outcome was.

And here's an example of an error, for us to see how it's passed through `finally` to `catch`:

```
new Promise((resolve, reject) => {
  throw new Error("error");
})
.finally(() => alert("Promise ready")) // triggers first
.catch(err => alert(err)); // <-- .catch shows the error
```

3. A `finally` handler also shouldn't return anything. If it does, the returned value is silently ignored.

The only exception to this rule is when a `finally` handler throws an error. Then this error goes to the next handler, instead of any previous outcome.

To summarize:

- A `finally` handler doesn't get the outcome of the previous handler (it has no arguments). This outcome is passed through instead, to the next suitable handler.
- If a `finally` handler returns something, it's ignored.
- When `finally` throws an error, then the execution goes to the nearest error handler.

These features are helpful and make things work just the right way if we use `finally` how it's supposed to be used: for generic cleanup procedures.

i We can attach handlers to settled promises

If a promise is pending, `.then/catch/finally` handlers wait for its outcome.

Sometimes, it might be that a promise is already settled when we add a handler to it.

In such case, these handlers just run immediately:

```
// the promise becomes resolved immediately upon creation
let promise = new Promise(resolve => resolve("done!"));

promise.then(alert); // done! (shows up right now)
```

Note that this makes promises more powerful than the real life “subscription list” scenario. If the singer has already released their song and then a person signs up on the subscription list, they probably won't receive that song. Subscriptions in real life must be done prior to the event.

Promises are more flexible. We can add handlers any time: if the result is already there, they just execute.

Example: loadScript

Next, let's see more practical examples of how promises can help us write asynchronous code.

We've got the `loadScript` function for loading a script from the previous chapter.

Here's the callback-based variant, just to remind us of it:

```
function loadScript(src, callback) {
  let script = document.createElement('script');
  script.src = src;

  script.onload = () => callback(null, script);
  script.onerror = () => callback(new Error(`Script load error for ${src}`));

  document.head.append(script);
}
```

Let's rewrite it using Promises.

The new function `loadScript` will not require a callback. Instead, it will create and return a Promise object that resolves when the loading is complete. The outer code can add handlers (subscribing functions) to it using `.then`:

```
function loadScript(src) {
  return new Promise(function(resolve, reject) {
    let script = document.createElement('script');
    script.src = src;

    script.onload = () => resolve(script);
    script.onerror = () => reject(new Error(`Script load error for ${src}`));

    document.head.append(script);
  });
}
```

Usage:

```
let promise = loadScript("https://cdnjs.cloudflare.com/ajax/libs/lodash.js/4.17.11/lodash.js");

promise.then(
  script => alert(`${script.src} is loaded!`),
  error => alert(`Error: ${error.message}`)
);

promise.then(script => alert('Another handler...'));
```

We can immediately see a few benefits over the callback-based pattern:

Promises	Callbacks
Promises allow us to do things in the natural order. First, we run <code>loadScript(script)</code> , and <code>.then</code> we write what to do with the result.	We must have a <code>callback</code> function at our disposal when calling <code>loadScript(script, callback)</code> . In other words, we must know what to do with the result <i>before</i> <code>loadScript</code> is called.
We can call <code>.then</code> on a Promise as many times as we want. Each time, we're adding a new "fan", a new subscribing function, to the "subscription list". More about this in the next chapter: Promises chaining .	There can be only one callback.

So promises give us better code flow and flexibility. But there's more. We'll see that in the next chapters.

✓ Tasks

Re-resolve a promise?

What's the output of the code below?

```
let promise = new Promise(function(resolve, reject) {
  resolve(1);

  setTimeout(() => resolve(2), 1000);
});

promise.then(alert);
```

[To solution](#)

Delay with a promise

The built-in function `setTimeout` uses callbacks. Create a promise-based alternative.

The function `delay(ms)` should return a promise. That promise should resolve after `ms` milliseconds, so that we can add `.then` to it, like this:

```
function delay(ms) {
  // your code
}

delay(3000).then(() => alert('runs after 3 seconds'));
```

[To solution](#)

Animated circle with promise

Rewrite the `showCircle` function in the solution of the task [Animated circle with callback](#) so that it returns a promise instead of accepting a callback.

The new usage:

```
showCircle(150, 150, 100).then(div => {
  div.classList.add('message-ball');
  div.append("Hello, world!");
});
```

Take the solution of the task [Animated circle with callback](#) as the base.

[To solution](#)

Promises chaining

Let's return to the problem mentioned in the chapter [Introduction: callbacks](#): we have a sequence of asynchronous tasks to be performed one after another — for instance, loading scripts. How can we code it well?

Promises provide a couple of recipes to do that.

In this chapter we cover promise chaining.

It looks like this:

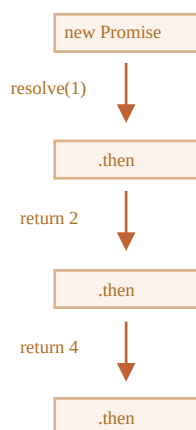
```
new Promise(function(resolve, reject) {  
  
  setTimeout(() => resolve(1), 1000); // (*)  
  
}).then(function(result) { // (**)  
  
  alert(result); // 1  
  return result * 2;  
  
}).then(function(result) { // (***)  
  
  alert(result); // 2  
  return result * 2;  
  
}).then(function(result) {  
  
  alert(result); // 4  
  return result * 2;  
  
});
```

The idea is that the result is passed through the chain of `.then` handlers.

Here the flow is:

1. The initial promise resolves in 1 second `(*)`,
2. Then the `.then` handler is called `(**)`, which in turn creates a new promise (resolved with `2` value).
3. The next `then` `(***)` gets the result of the previous one, processes it (doubles) and passes it to the next handler.
4. ...and so on.

As the result is passed along the chain of handlers, we can see a sequence of `alert` calls: `1 → 2 → 4`.



The whole thing works, because every call to a `.then` returns a new promise, so that we can call the next `.then` on it.

When a handler returns a value, it becomes the result of that promise, so the next `.then` is called with it.

A classic newbie error: technically we can also add many `.then` to a single promise. This is not chaining.

For example:

```
let promise = new Promise(function(resolve, reject) {  
  setTimeout(() => resolve(1), 1000);  
});
```

```

promise.then(function(result) {
  alert(result); // 1
  return result * 2;
});

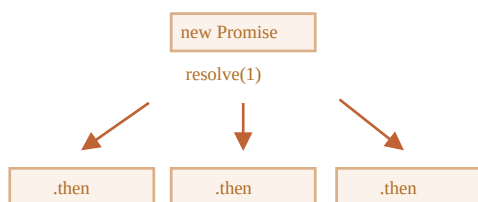
promise.then(function(result) {
  alert(result); // 1
  return result * 2;
});

promise.then(function(result) {
  alert(result); // 1
  return result * 2;
});

```

What we did here is just adding several handlers to one promise. They don't pass the result to each other; instead they process it independently.

Here's the picture (compare it with the chaining above):



All `.then` on the same promise get the same result – the result of that promise. So in the code above all `alert` show the same: 1.

In practice we rarely need multiple handlers for one promise. Chaining is used much more often.

Returning promises

A handler, used in `.then(handler)` may create and return a promise.

In that case further handlers wait until it settles, and then get its result.

For instance:

```

new Promise(function(resolve, reject) {
  setTimeout(() => resolve(1), 1000);
}).then(function(result) {
  alert(result); // 1

  return new Promise((resolve, reject) => { // (*)
    setTimeout(() => resolve(result * 2), 1000);
  });
}).then(function(result) { // (**)
  alert(result); // 2

  return new Promise((resolve, reject) => {
    setTimeout(() => resolve(result * 2), 1000);
  });
}).then(function(result) {
  alert(result); // 4
});

```

Here the first `.then` shows 1 and returns `new Promise(...)` in the line `(*)`. After one second it resolves, and the result (the argument of `resolve`, here it's `result * 2`) is passed on to the handler of the second `.then`. That handler is in the line `(**)`, it shows 2 and does the same thing.

So the output is the same as in the previous example: 1 → 2 → 4, but now with 1 second delay between `alert` calls.

Returning promises allows us to build chains of asynchronous actions.

Example: loadScript

Let's use this feature with the promisified `loadScript`, defined in the [previous chapter](#), to load scripts one by one, in sequence:

```
loadScript("/article/promise-chaining/one.js")
  .then(function(script) {
    return loadScript("/article/promise-chaining/two.js");
  })
  .then(function(script) {
    return loadScript("/article/promise-chaining/three.js");
  })
  .then(function(script) {
    // use functions declared in scripts
    // to show that they indeed loaded
    one();
    two();
    three();
  });
```

This code can be made bit shorter with arrow functions:

```
loadScript("/article/promise-chaining/one.js")
  .then(script => loadScript("/article/promise-chaining/two.js"))
  .then(script => loadScript("/article/promise-chaining/three.js"))
  .then(script => {
    // scripts are loaded, we can use functions declared there
    one();
    two();
    three();
  });
```

Here each `loadScript` call returns a promise, and the next `.then` runs when it resolves. Then it initiates the loading of the next script. So scripts are loaded one after another.

We can add more asynchronous actions to the chain. Please note that the code is still “flat” — it grows down, not to the right. There are no signs of the “pyramid of doom”.

Technically, we could add `.then` directly to each `loadScript`, like this:

```
loadScript("/article/promise-chaining/one.js").then(script1 => {
  loadScript("/article/promise-chaining/two.js").then(script2 => {
    loadScript("/article/promise-chaining/three.js").then(script3 => {
      // this function has access to variables script1, script2 and script3
      one();
      two();
      three();
    });
  });
});
```

This code does the same: loads 3 scripts in sequence. But it “grows to the right”. So we have the same problem as with callbacks.

People who start to use promises sometimes don't know about chaining, so they write it this way. Generally, chaining is preferred.

Sometimes it's ok to write `.then` directly, because the nested function has access to the outer scope. In the example above the most nested callback has access to all variables `script1`, `script2`, `script3`. But that's an exception rather than a rule.

i Thenables

To be precise, a handler may return not exactly a promise, but a so-called “thenable” object – an arbitrary object that has a method `.then`. It will be treated the same way as a promise.

The idea is that 3rd-party libraries may implement “promise-compatible” objects of their own. They can have an extended set of methods, but also be compatible with native promises, because they implement `.then`.

Here's an example of a thenable object:

```
class Thenable {
  constructor(num) {
    this.num = num;
  }
  then(resolve, reject) {
    alert(resolve); // function() { native code }
    // resolve with this.num*2 after the 1 second
    setTimeout(() => resolve(this.num * 2), 1000); // (**)
  }
}

new Promise(resolve => resolve(1))
  .then(result => {
    return new Thenable(result); // (*)
  })
  .then(alert); // shows 2 after 1000ms
```

JavaScript checks the object returned by the `.then` handler in line `(*)`: if it has a callable method named `then`, then it calls that method providing native functions `resolve`, `reject` as arguments (similar to an executor) and waits until one of them is called. In the example above `resolve(2)` is called after 1 second `(**)`. Then the result is passed further down the chain.

This feature allows us to integrate custom objects with promise chains without having to inherit from `Promise`.

Bigger example: fetch

In frontend programming, promises are often used for network requests. So let's see an extended example of that.

We'll use the `fetch` method to load the information about the user from the remote server. It has a lot of optional parameters covered in [separate chapters](#), but the basic syntax is quite simple:

```
let promise = fetch(url);
```

This makes a network request to the `url` and returns a promise. The promise resolves with a `response` object when the remote server responds with headers, but *before the full response is downloaded*.

To read the full response, we should call the method `response.text()`: it returns a promise that resolves when the full text is downloaded from the remote server, with that text as a result.

The code below makes a request to `user.json` and loads its text from the server:

```
fetch('/article/promise-chaining/user.json')
  // .then below runs when the remote server responds
  .then(function(response) {
    // response.text() returns a new promise that resolves with the full response text
    // when it loads
    return response.text();
  })
  .then(function(text) {
    // ...and here's the content of the remote file
    alert(text); // {"name": "iliakan", "isAdmin": true}
  });
```

The `response` object returned from `fetch` also includes the method `response.json()` that reads the remote data and parses it as JSON. In our case that's even more convenient, so let's switch to it.

We'll also use arrow functions for brevity:

```
// same as above, but response.json() parses the remote content as JSON
fetch('/article/promise-chaining/user.json')
  .then(response => response.json())
  .then(user => alert(user.name)); // ilikan, got user name
```

Now let's do something with the loaded user.

For instance, we can make one more request to GitHub, load the user profile and show the avatar:

```
// Make a request for user.json
fetch('/article/promise-chaining/user.json')
  // Load it as json
  .then(response => response.json())
  // Make a request to GitHub
  .then(user => fetch(`https://api.github.com/users/${user.name}`))
  // Load the response as json
  .then(response => response.json())
  // Show the avatar image (githubUser.avatar_url) for 3 seconds (maybe animate it)
  .then(githubUser => {
    let img = document.createElement('img');
    img.src = githubUser.avatar_url;
    img.className = "promise-avatar-example";
    document.body.append(img);

    setTimeout(() => img.remove(), 3000); // (*)
  });
```

The code works; see comments about the details. However, there's a potential problem in it, a typical error for those who begin to use promises.

Look at the line `(*)`: how can we do something *after* the avatar has finished showing and gets removed? For instance, we'd like to show a form for editing that user or something else. As of now, there's no way.

To make the chain extendable, we need to return a promise that resolves when the avatar finishes showing.

Like this:

```
fetch('/article/promise-chaining/user.json')
  .then(response => response.json())
  .then(user => fetch(`https://api.github.com/users/${user.name}`))
  .then(response => response.json())
  .then(githubUser => new Promise(function(resolve, reject) { // (*)
    let img = document.createElement('img');
    img.src = githubUser.avatar_url;
    img.className = "promise-avatar-example";
    document.body.append(img);

    setTimeout(() => {
      img.remove();
      resolve(githubUser); // (**)
    }, 3000);
  })))
  // triggers after 3 seconds
  .then(githubUser => alert(`Finished showing ${githubUser.name}`));
```

That is, the `.then` handler in line `(*)` now returns `new Promise`, that becomes settled only after the call of `resolve(githubUser)` in `setTimeout (**)`. The next `.then` in the chain will wait for that.

As a good practice, an asynchronous action should always return a promise. That makes it possible to plan actions after it; even if we don't plan to extend the chain now, we may need it later.

Finally, we can split the code into reusable functions:

```
function loadJson(url) {
  return fetch(url)
    .then(response => response.json());
}

function loadGithubUser(name) {
  return loadJson(`https://api.github.com/users/${name}`);
}
```

```

}

function showAvatar(githubUser) {
  return new Promise(function(resolve, reject) {
    let img = document.createElement('img');
    img.src = githubUser.avatar_url;
    img.className = "promise-avatar-example";
    document.body.append(img);

    setTimeout(() => {
      img.remove();
      resolve(githubUser);
    }, 3000);
  });
}

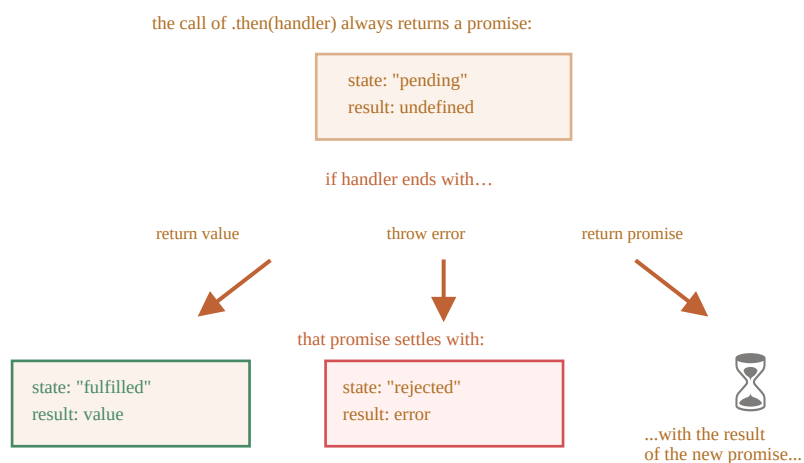
// Use them:
loadJson('/article/promise-chaining/user.json')
  .then(user => loadGithubUser(user.name))
  .then(showAvatar)
  .then(githubUser => alert(`Finished showing ${githubUser.name}`));
// ...

```

Summary

If a `.then` (or `catch/finally`, doesn't matter) handler returns a promise, the rest of the chain waits until it settles. When it does, its result (or error) is passed further.

Here's a full picture:



✓ Tasks

Promise: then versus catch

Are these code fragments equal? In other words, do they behave the same way in any circumstances, for any handler functions?

```
promise.then(f1).catch(f2);
```

Versus:

```
promise.then(f1, f2);
```

[To solution](#)

Error handling with promises

Promise chains are great at error handling. When a promise rejects, the control jumps to the closest rejection handler. That's very convenient in practice.

For instance, in the code below the URL to `fetch` is wrong (no such site) and `.catch` handles the error:

```
fetch('https://no-such-server.blabla') // rejects
  .then(response => response.json())
  .catch(err => alert(err)) // TypeError: failed to fetch (the text may vary)
```

As you can see, the `.catch` doesn't have to be immediate. It may appear after one or maybe several `.then`.

Or, maybe, everything is all right with the site, but the response is not valid JSON. The easiest way to catch all errors is to append `.catch` to the end of chain:

```
fetch('/article/promise-chaining/user.json')
  .then(response => response.json())
  .then(user => fetch(`https://api.github.com/users/${user.name}`))
  .then(response => response.json())
  .then(githubUser => new Promise((resolve, reject) => {
    let img = document.createElement('img');
    img.src = githubUser.avatar_url;
    img.className = "promise-avatar-example";
    document.body.append(img);

    setTimeout(() => {
      img.remove();
      resolve(githubUser);
    }, 3000);
  })))
  .catch(error => alert(error.message));
```

Normally, such `.catch` doesn't trigger at all. But if any of the promises above rejects (a network problem or invalid json or whatever), then it would catch it.

Implicit try...catch

The code of a promise executor and promise handlers has an “invisible `try...catch`” around it. If an exception happens, it gets caught and treated as a rejection.

For instance, this code:

```
new Promise((resolve, reject) => {
  throw new Error("Whoops!");
}).catch(alert); // Error: Whoops!
```

...Works exactly the same as this:

```
new Promise((resolve, reject) => {
  reject(new Error("Whoops!"));
}).catch(alert); // Error: Whoops!
```

The “invisible `try...catch`” around the executor automatically catches the error and turns it into rejected promise.

This happens not only in the executor function, but in its handlers as well. If we `throw` inside a `.then` handler, that means a rejected promise, so the control jumps to the nearest error handler.

Here's an example:

```
new Promise((resolve, reject) => {
  resolve("ok");
}).then((result) => {
  throw new Error("Whoops!"); // rejects the promise
}).catch(alert); // Error: Whoops!
```

This happens for all errors, not just those caused by the `throw` statement. For example, a programming error:

```
new Promise((resolve, reject) => {
  resolve("ok");
}).then((result) => {
  blabla(); // no such function
}).catch(alert); // ReferenceError: blabla is not defined
```

The final `.catch` not only catches explicit rejections, but also accidental errors in the handlers above.

Rethrowing

As we already noticed, `.catch` at the end of the chain is similar to `try..catch`. We may have as many `.then` handlers as we want, and then use a single `.catch` at the end to handle errors in all of them.

In a regular `try..catch` we can analyze the error and maybe rethrow it if it can't be handled. The same thing is possible for promises.

If we `throw` inside `.catch`, then the control goes to the next closest error handler. And if we handle the error and finish normally, then it continues to the next closest successful `.then` handler.

In the example below the `.catch` successfully handles the error:

```
// the execution: catch -> then
new Promise((resolve, reject) => {

  throw new Error("Whoops!");

}).catch(function(error) {

  alert("The error is handled, continue normally");

}).then(() => alert("Next successful handler runs"));
```

Here the `.catch` block finishes normally. So the next successful `.then` handler is called.

In the example below we see the other situation with `.catch`. The handler `(*)` catches the error and just can't handle it (e.g. it only knows how to handle `URIError`), so it throws it again:

```
// the execution: catch -> catch
new Promise((resolve, reject) => {

  throw new Error("Whoops!");

}).catch(function(error) { // (*)

  if (error instanceof URIError) {
    // handle it
  } else {
    alert("Can't handle such error");

    throw error; // throwing this or another error jumps to the next catch
  }

}).then(function() {
  /* doesn't run here */
}).catch(error => { // (**)

  alert(`The unknown error has occurred: ${error}`);
  // don't return anything => execution goes the normal way

});
```

The execution jumps from the first `.catch (*)` to the next one `(**)` down the chain.

Unhandled rejections

What happens when an error is not handled? For instance, we forgot to append `.catch` to the end of the chain, like here:

```
new Promise(function() {
  noSuchFunction(); // Error here (no such function)
})
.then(() => {
  // successful promise handlers, one or more
}); // without .catch at the end!
```

In case of an error, the promise becomes rejected, and the execution should jump to the closest rejection handler. But there is none. So the error gets “stuck”. There’s no code to handle it.

In practice, just like with regular unhandled errors in code, it means that something has gone terribly wrong.

What happens when a regular error occurs and is not caught by `try...catch`? The script dies with a message in the console. A similar thing happens with unhandled promise rejections.

The JavaScript engine tracks such rejections and generates a global error in that case. You can see it in the console if you run the example above.

In the browser we can catch such errors using the event `unhandledrejection`:

```
window.addEventListener('unhandledrejection', function(event) {
  // the event object has two special properties:
  alert(event.promise); // [object Promise] - the promise that generated the error
  alert(event.reason); // Error: Whoops! - the unhandled error object
});

new Promise(function() {
  throw new Error("Whoops!");
}); // no catch to handle the error
```

The event is the part of the [HTML standard](#).

If an error occurs, and there’s no `.catch`, the `unhandledrejection` handler triggers, and gets the `event` object with the information about the error, so we can do something.

Usually such errors are unrecoverable, so our best way out is to inform the user about the problem and probably report the incident to the server.

In non-browser environments like Node.js there are other ways to track unhandled errors.

Summary

- `.catch` handles errors in promises of all kinds: be it a `reject()` call, or an error thrown in a handler.
- `.then` also catches errors in the same manner, if given the second argument (which is the error handler).
- We should place `.catch` exactly in places where we want to handle errors and know how to handle them. The handler should analyze errors (custom error classes help) and rethrow unknown ones (maybe they are programming mistakes).
- It’s ok not to use `.catch` at all, if there’s no way to recover from an error.
- In any case we should have the `unhandledrejection` event handler (for browsers, and analogs for other environments) to track unhandled errors and inform the user (and probably our server) about them, so that our app never “just dies”.

✓ Tasks

Error in `setTimeout`

What do you think? Will the `.catch` trigger? Explain your answer.

```
new Promise(function(resolve, reject) {
  setTimeout(() => {
    throw new Error("Whoops!");
  }, 1000);
}).catch(alert);
```

To solution

```
let promise = Promise.reject(new Error("Promise Failed!"));
setTimeout(() => promise.catch(err => alert('caught')), 1000);

// Error: Promise Failed!
window.addEventListener('unhandledrejection', event => alert(event.reason));
```

Now, if we run it, we'll see `Promise Failed!` first and then `caught`.

If we didn't know about the microtasks queue, we could wonder: "Why did `unhandledrejection` handler run? We did catch and handle the error!"

But now we understand that `unhandledrejection` is generated when the microtask queue is complete: the engine examines promises and, if any of them is in the "rejected" state, then the event triggers.

In the example above, `.catch` added by `setTimeout` also triggers. But it does so later, after `unhandledrejection` has already occurred, so it doesn't change anything.

Summary

Promise handling is always asynchronous, as all promise actions pass through the internal "promise jobs" queue, also called "microtask queue" (V8 term).

So `.then/catch/finally` handlers are always called after the current code is finished.

If we need to guarantee that a piece of code is executed after `.then/catch/finally`, we can add it into a chained `.then` call.

In most Javascript engines, including browsers and Node.js, the concept of microtasks is closely tied with the "event loop" and "macrotasks". As these have no direct relation to promises, they are covered in another part of the tutorial, in the article [Event loop: microtasks and macrotasks](#).

Async/await

There's a special syntax to work with promises in a more comfortable fashion, called "async/await". It's surprisingly easy to understand and use.

Async functions

Let's start with the `async` keyword. It can be placed before a function, like this:

```
async function f() {
  return 1;
}
```

The word "async" before a function means one simple thing: a function always returns a promise. Other values are wrapped in a resolved promise automatically.

For instance, this function returns a resolved promise with the result of `1`; let's test it:

```
async function f() {
  return 1;
}

f().then(alert); // 1
```

...We could explicitly return a promise, which would be the same:

```
async function f() {
  return Promise.resolve(1);
}

f().then(alert); // 1
```

So, `async` ensures that the function returns a promise, and wraps non-promises in it. Simple enough, right? But not only that. There's another keyword, `await`, that works only inside `async` functions, and it's pretty cool.

Await

The syntax:

```
// works only inside async functions
let value = await promise;
```

The keyword `await` makes JavaScript wait until that promise settles and returns its result.

Here's an example with a promise that resolves in 1 second:

```
async function f() {

  let promise = new Promise((resolve, reject) => {
    setTimeout(() => resolve("done!"), 1000)
  });

  let result = await promise; // wait until the promise resolves (*)

  alert(result); // "done!"
}

f();
```

The function execution “pauses” at the line `(*)` and resumes when the promise settles, with `result` becoming its result. So the code above shows “done!” in one second.

Let's emphasize: `await` literally suspends the function execution until the promise settles, and then resumes it with the promise result. That doesn't cost any CPU resources, because the JavaScript engine can do other jobs in the meantime: execute other scripts, handle events, etc.

It's just a more elegant syntax of getting the promise result than `promise.then`. And, it's easier to read and write.

⚠ Can't use `await` in regular functions

If we try to use `await` in a non-async function, there would be a syntax error:

```
function f() {
  let promise = Promise.resolve(1);
  let result = await promise; // Syntax error
}
```

We may get this error if we forget to put `async` before a function. As stated earlier, `await` only works inside an `async` function.

Let's take the `showAvatar()` example from the chapter [Promises chaining](#) and rewrite it using `async/await`:

1. We'll need to replace `.then` calls with `await`.
2. Also we should make the function `async` for them to work.

```
async function showAvatar() {

  // read our JSON
  let response = await fetch('/article/promise-chaining/user.json');
  let user = await response.json();

  // read github user
  let githubResponse = await fetch(`https://api.github.com/users/${user.name}`);
  let githubUser = await githubResponse.json();

  // show the avatar
  let img = document.createElement('img');
  img.src = githubUser.avatar_url;
  img.className = "promise-avatar-example";
  document.body.append(img);

  // wait 3 seconds
```

```

    await new Promise((resolve, reject) => setTimeout(resolve, 3000));

    img.remove();

    return githubUser;
}

showAvatar();

```

Pretty clean and easy to read, right? Much better than before.

i Modern browsers allow top-level `await` in modules

In modern browsers, `await` on top level works just fine, when we're inside a module. We'll cover modules in article [Modules, introduction](#).

For instance:

```

// we assume this code runs at top level, inside a module
let response = await fetch('/article/promise-chaining/user.json');
let user = await response.json();

console.log(user);

```

If we're not using modules, or [older browsers](#) must be supported, there's a universal recipe: wrapping into an anonymous async function.

Like this:

```

(async () => {
  let response = await fetch('/article/promise-chaining/user.json');
  let user = await response.json();
  ...
})();

```

i `await` accepts "thenables"

Like `promise.then`, `await` allows us to use thenable objects (those with a callable `then` method). The idea is that a third-party object may not be a promise, but promise-compatible: if it supports `.then`, that's enough to use it with `await`.

Here's a demo `Thenable` class; the `await` below accepts its instances:

```

class Thenable {
  constructor(num) {
    this.num = num;
  }
  then(resolve, reject) {
    alert(resolve);
    // resolve with this.num*2 after 1000ms
    setTimeout(() => resolve(this.num * 2), 1000); // (*)
  }
}

async function f() {
  // waits for 1 second, then result becomes 2
  let result = await new Thenable(1);
  alert(result);
}

f();

```

If `await` gets a non-promise object with `.then`, it calls that method providing the built-in functions `resolve` and `reject` as arguments (just as it does for a regular `Promise` executor). Then `await` waits until one of them is called (in the example above it happens in the line `(*)`) and then proceeds with the result.

Async class methods

To declare an async class method, just prepend it with `async`:

```
class Waiter {  
  async wait() {  
    return await Promise.resolve(1);  
  }  
}  
  
new Waiter()  
  .wait()  
  .then(alert); // 1 (this is the same as (result => alert(result)))
```

The meaning is the same: it ensures that the returned value is a promise and enables `await`.

Error handling

If a promise resolves normally, then `await promise` returns the result. But in the case of a rejection, it throws the error, just as if there were a `throw` statement at that line.

This code:

```
async function f() {  
  await Promise.reject(new Error("Whoops!"));  
}
```

...is the same as this:

```
async function f() {  
  throw new Error("Whoops!");  
}
```

In real situations, the promise may take some time before it rejects. In that case there will be a delay before `await` throws an error.

We can catch that error using `try...catch`, the same way as a regular `throw`:

```
async function f() {  
  try {  
    let response = await fetch('http://no-such-url');  
  } catch(err) {  
    alert(err); // TypeError: failed to fetch  
  }  
}  
  
f();
```

In the case of an error, the control jumps to the `catch` block. We can also wrap multiple lines:

```
async function f() {  
  try {  
    let response = await fetch('/no-user-here');  
    let user = await response.json();  
  } catch(err) {  
    // catches errors both in fetch and response.json  
    alert(err);  
  }  
}  
  
f();
```

If we don't have `try..catch`, then the promise generated by the call of the async function `f()` becomes rejected. We can append `.catch` to handle it:

```
async function f() {
  let response = await fetch('http://no-such-url');
}

// f() becomes a rejected promise
f().catch(alert); // TypeError: failed to fetch // (*)
```

If we forget to add `.catch` there, then we get an unhandled promise error (viewable in the console). We can catch such errors using a global `unhandledrejection` event handler as described in the chapter [Error handling with promises](#).

async/await and promise.then/catch

When we use `async/await`, we rarely need `.then`, because `await` handles the waiting for us. And we can use a regular `try..catch` instead of `.catch`. That's usually (but not always) more convenient.

But at the top level of the code, when we're outside any `async` function, we're syntactically unable to use `await`, so it's a normal practice to add `.then/catch` to handle the final result or falling-through error, like in the line `(*)` of the example above.

async/await works well with Promise.all

When we need to wait for multiple promises, we can wrap them in `Promise.all` and then `await`:

```
// wait for the array of results
let results = await Promise.all([
  fetch(url1),
  fetch(url2),
  ...
]);
```

In the case of an error, it propagates as usual, from the failed promise to `Promise.all`, and then becomes an exception that we can catch using `try..catch` around the call.

Summary

The `async` keyword before a function has two effects:

1. Makes it always return a promise.
2. Allows `await` to be used in it.

The `await` keyword before a promise makes JavaScript wait until that promise settles, and then:

1. If it's an error, an exception is generated — same as if `throw error` were called at that very place.
2. Otherwise, it returns the result.

Together they provide a great framework to write asynchronous code that is easy to both read and write.

With `async/await` we rarely need to write `promise.then/catch`, but we still shouldn't forget that they are based on promises, because sometimes (e.g. in the outermost scope) we have to use these methods. Also `Promise.all` is nice when we are waiting for many tasks simultaneously.

Tasks

Rewrite using async/await

Rewrite this example code from the chapter [Promises chaining](#) using `async/await` instead of `.then/catch`:

```
function loadJson(url) {
  return fetch(url)
    .then(response => {
      if (response.status == 200) {
```

```

        return response.json();
    } else {
        throw new Error(response.status);
    }
    });
}

loadJson('https://javascript.info/no-such-user.json')
    .catch(alert); // Error: 404

```

To solution

Rewrite "rethrow" with async/await

Below you can find the “rethrow” example. Rewrite it using `async/await` instead of `.then/catch`.

And get rid of the recursion in favour of a loop in `demoGithubUser`: with `async/await` that becomes easy to do.

```

class HttpError extends Error {
  constructor(response) {
    super(`${response.status} for ${response.url}`);
    this.name = 'HttpError';
    this.response = response;
  }
}

function loadJson(url) {
  return fetch(url)
    .then(response => {
      if (response.status == 200) {
        return response.json();
      } else {
        throw new HttpError(response);
      }
    });
}

// Ask for a user name until github returns a valid user
function demoGithubUser() {
  let name = prompt("Enter a name?", "iliakan");

  return loadJson(`https://api.github.com/users/${name}`)
    .then(user => {
      alert(`Full name: ${user.name}.`);
      return user;
    })
    .catch(err => {
      if (err instanceof HttpError && err.response.status == 404) {
        alert("No such user, please reenter.");
        return demoGithubUser();
      } else {
        throw err;
      }
    });
}

demoGithubUser();

```

To solution

Call async from non-async

We have a “regular” function called `f`. How can you call the `async` function `wait()` and use its result inside of `f`?

```

async function wait() {
  await new Promise(resolve => setTimeout(resolve, 1000));

  return 10;
}

function f() {

```

```
// ...what should you write here?
// we need to call async wait() and wait to get 10
// remember, we can't use "await"
}
```

P.S. The task is technically very simple, but the question is quite common for developers new to async/await.

[To solution](#)

Dangerous Promise.all

`Promise.all` is a great way to parallelize multiple operations. It's especially useful when we need to make parallel requests to multiple services.

However, there's a hidden danger. We'll see an example in this task and explore how to avoid it.

Let's say we have a connection to a remote service, such as a database.

There're two functions: `connect()` and `disconnect()`.

When connected, we can send requests using `database.query(...)` – an async function which usually returns the result but also may throw an error.

Here's a simple implementation:

```
let database;

function connect() {
  database = {
    async query(isOk) {
      if (!isOk) throw new Error('Query failed');
    }
  };
};

function disconnect() {
  database = null;
}

// intended usage:
// connect()
// ...
// database.query(true) to emulate a successful call
// database.query(false) to emulate a failed call
// ...
// disconnect()
```

Now here's the problem.

We wrote the code to connect and send 3 queries in parallel (all of them take different time, e.g. 100, 200 and 300ms), then disconnect:

```
// Helper function to call async function `fn` after `ms` milliseconds
function delay(fn, ms) {
  return new Promise((resolve, reject) => {
    setTimeout(() => fn().then(resolve, reject), ms);
  });
}

async function run() {
  connect();

  try {
    await Promise.all([
      // these 3 parallel jobs take different time: 100, 200 and 300 ms
      // we use the `delay` helper to achieve this effect
      delay(() => database.query(true), 100),
      delay(() => database.query(false), 200),
      delay(() => database.query(false), 300)
    ]);
  }
}
```

Network requests

Fetch

JavaScript can send network requests to the server and load new information whenever it's needed.

For example, we can use a network request to:

- Submit an order,
- Load user information,
- Receive latest updates from the server,
- ...etc.

...And all of that without reloading the page!

There's an umbrella term "AJAX" (abbreviated **A**synchronous **J**avaScript **A**nd **X**ML) for network requests from JavaScript. We don't have to use XML though: the term comes from old times, that's why that word is there. You may have heard that term already.

There are multiple ways to send a network request and get information from the server.

The `fetch()` method is modern and versatile, so we'll start with it. It's not supported by old browsers (can be polyfilled), but very well supported among the modern ones.

The basic syntax is:

```
let promise = fetch(url, [options])
```

- `url` – the URL to access.
- `options` – optional parameters: method, headers etc.

Without `options`, this is a simple GET request, downloading the contents of the `url`.

The browser starts the request right away and returns a promise that the calling code should use to get the result.

Getting a response is usually a two-stage process.

First, the `promise`, returned by `fetch`, resolves with an object of the built-in `Response` [class](#) as soon as the server responds with headers.

At this stage we can check HTTP status, to see whether it is successful or not, check headers, but don't have the body yet.

The promise rejects if the `fetch` was unable to make HTTP-request, e.g. network problems, or there's no such site. Abnormal HTTP-statuses, such as 404 or 500 do not cause an error.

We can see HTTP-status in response properties:

- `status` – HTTP status code, e.g. 200.
- `ok` – boolean, `true` if the HTTP status code is 200-299.

For example:

```
let response = await fetch(url);

if (response.ok) { // if HTTP-status is 200-299
  // get the response body (the method explained below)
  let json = await response.json();
} else {
  alert("HTTP-Error: " + response.status);
}
```

Second, to get the response body, we need to use an additional method call.

`Response` provides multiple promise-based methods to access the body in various formats:

- `response.text()` – read the response and return as text,
- `response.json()` – parse the response as JSON,
- `response.formData()` – return the response as `FormData` object (explained in the [next chapter](#)),
- `response.blob()` – return the response as `Blob` (binary data with type),
- `response.arrayBuffer()` – return the response as `ArrayBuffer` (low-level representation of binary data),
- additionally, `response.body` is a [ReadableStream](#) object, it allows you to read the body chunk-by-chunk, we'll see an example later.

For instance, let's get a JSON-object with latest commits from GitHub:

```
let url = 'https://api.github.com/repos/javascript-tutorial/en.javascript.info'
let response = await fetch(url);

let commits = await response.json(); // read response body and parse as JSON

alert(commits[0].author.login);
```

Or, the same without `await`, using pure promises syntax:

```
fetch('https://api.github.com/repos/javascript-tutorial/en.javascript.info/commits')
  .then(response => response.json())
  .then(commits => alert(commits[0].author.login));
```

To get the response text, `await response.text()` instead of `.json()`:

```
let response = await fetch('https://api.github.com/repos/javascript-tutorial/en.javascript.info/commits')

let text = await response.text(); // read response body as text

alert(text.slice(0, 80) + '...');
```

As a show-case for reading in binary format, let's fetch and show a logo image of “[fetch](#)” specification (see chapter [Blob](#) for details about operations on `Blob`):

```
let response = await fetch('/article/fetch/logo-fetch.svg');

let blob = await response.blob(); // download as Blob object
```

```
// create <img> for it
let img = document.createElement('img');
img.style = 'position:fixed;top:10px;left:10px;width:100px';
document.body.append(img);

// show it
img.src = URL.createObjectURL(blob);

setTimeout(() => { // hide after three seconds
  img.remove();
  URL.revokeObjectURL(img.src);
}, 3000);
```

Important:

We can choose only one body-reading method.

If we've already got the response with `response.text()`, then `response.json()` won't work, as the body content has already been processed.

```
let text = await response.text(); // response body consumed
let parsed = await response.json(); // fails (already consumed)
```

Response headers

The response headers are available in a Map-like headers object in `response.headers`.

It's not exactly a Map, but it has similar methods to get individual headers by name or iterate over them:

```
let response = await fetch('https://api.github.com/repos/javascript-tutorial/e

// get one header
alert(response.headers.get('Content-Type')); // application/json; charset=utf-

// iterate over all headers
for (let [key, value] of response.headers) {
  alert(`${key} = ${value}`);
}
```

Request headers

To set a request header in `fetch`, we can use the `headers` option. It has an object with outgoing headers, like this:

```
let response = fetch(protectedUrl, {
  headers: {
    Authentication: 'secret'
  }
});
```

...But there's a list of [forbidden HTTP headers](#) that we can't set:

- Accept-Charset, Accept-Encoding
- Access-Control-Request-Headers
- Access-Control-Request-Method
- Connection
- Content-Length
- Cookie, Cookie2
- Date
- DNT
- Expect
- Host
- Keep-Alive
- Origin
- Referer
- TE
- Trailer
- Transfer-Encoding
- Upgrade
- Via
- Proxy-*
- Sec-*

These headers ensure proper and safe HTTP, so they are controlled exclusively by the browser.

POST requests

To make a `POST` request, or a request with another method, we need to use `fetch` options:

- **method** – HTTP-method, e.g. `POST`,
- **body** – the request body, one of:
 - a string (e.g. JSON-encoded),
 - `FormData` object, to submit the data as `multipart/form-data`,
 - `Blob` / `BufferSource` to send binary data,
 - `URLSearchParams`, to submit the data in `x-www-form-urlencoded` encoding, rarely used.

The JSON format is used most of the time.

For example, this code submits `user` object as JSON:

```
let user = {
  name: 'John',
  surname: 'Smith'
};

let response = await fetch('/article/fetch/post/user', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json;charset=utf-8'
  },
  body: JSON.stringify(user)
});

let result = await response.json();
alert(result.message);
```

Please note, if the request `body` is a string, then `Content-Type` header is set to `text/plain;charset=UTF-8` by default.

But, as we're going to send JSON, we use `headers` option to send `application/json` instead, the correct `Content-Type` for JSON-encoded data.

Sending an image

We can also submit binary data with `fetch` using `Blob` or `BufferSource` objects.

In this example, there's a `<canvas>` where we can draw by moving a mouse over it. A click on the “submit” button sends the image to the server:

```

<body style="margin:0">
  <canvas id="canvasElem" width="100" height="80" style="border:1px solid"></c

  <input type="button" value="Submit" onclick="submit()">

  <script>
    canvasElem.onmousemove = function(e) {
      let ctx = canvasElem.getContext('2d');
      ctx.lineTo(e.clientX, e.clientY);
      ctx.stroke();
    };

    async function submit() {
      let blob = await new Promise(resolve => canvasElem.toBlob(resolve, 'imag
      let response = await fetch('/article/fetch/post/image', {
        method: 'POST',
        body: blob
      });

      // the server responds with confirmation and the image size
      let result = await response.json();
      alert(result.message);
    }

  </script>
</body>

```



Please note, here we don't set `Content-Type` header manually, because a `Blob` object has a built-in type (here `image/png`, as generated by `toBlob`). For `Blob` objects that type becomes the value of `Content-Type`.

The `submit()` function can be rewritten without `async/await` like this:

```

function submit() {
  canvasElem.toBlob(function(blob) {
    fetch('/article/fetch/post/image', {
      method: 'POST',
      body: blob
    })
    .then(response => response.json())
    .then(result => alert(JSON.stringify(result, null, 2)))
  }, 'image/png');
}

```

Summary

A typical fetch request consists of two `await` calls:

```
let response = await fetch(url, options); // resolves with response headers
let result = await response.json(); // read body as json
```

Or, without `await`:

```
fetch(url, options)
  .then(response => response.json())
  .then(result => /* process result */)
```

Response properties:

- `response.status` – HTTP code of the response,
- `response.ok` – `true` if the status is 200-299.
- `response.headers` – Map-like object with HTTP headers.

Methods to get response body:

- `response.text()` – return the response as text,
- `response.json()` – parse the response as JSON object,
- `response.formData()` – return the response as `FormData` object (multipart/form-data encoding, see the next chapter),
- `response.blob()` – return the response as `Blob` (binary data with type),
- `response.arrayBuffer()` – return the response as `ArrayBuffer` (low-level binary data),

Fetch options so far:

- `method` – HTTP-method,
- `headers` – an object with request headers (not any header is allowed),
- `body` – the data to send (request body) as `string`, `FormData`, `BufferSource`, `Blob` or `UrlSearchParams` object.

In the next chapters we'll see more options and use cases of `fetch`.

✓ Tasks

Fetch users from GitHub

Create an async function `getUsers(names)`, that gets an array of GitHub logins, fetches the users from GitHub and returns an array of GitHub users.

The GitHub url with user information for the given `USERNAME` is:

`https://api.github.com/users/USERNAME`.

There's a test example in the sandbox.

Important details:

1. There should be one `fetch` request per user.
2. Requests shouldn't wait for each other. So that the data arrives as soon as possible.
3. If any request fails, or if there's no such user, the function should return `null` in the resulting array.

[Open a sandbox with tests.](#) ➦

[To solution](#)

FormData

This chapter is about sending HTML forms: with or without files, with additional fields and so on.

[FormData](#) ➦ objects can help with that. As you might have guessed, it's the object to represent HTML form data.

The constructor is:

```
let formData = new FormData([form]);
```

If HTML `form` element is provided, it automatically captures its fields.

The special thing about `FormData` is that network methods, such as `fetch`, can accept a `FormData` object as a body. It's encoded and sent out with `Content-Type: multipart/form-data`.

From the server point of view, that looks like a usual form submission.

Sending a simple form

Let's send a simple form first.

As you can see, that's almost one-liner:

```
<form id="formElem">
  <input type="text" name="name" value="John">
  <input type="text" name="surname" value="Smith">
  <input type="submit">
</form>

<script>
  formElem.onsubmit = async (e) => {
    e.preventDefault();

    let response = await fetch('/article/formdata/post/user', {
      method: 'POST',
      body: new FormData(formElem)
    });

    let result = await response.json();

    alert(result.message);
  };
</script>
```

John Smith

In this example, the server code is not presented, as it's beyond our scope. The server accepts the POST request and replies "User saved".

FormData Methods

We can modify fields in `FormData` with methods:

- `formData.append(name, value)` – add a form field with the given `name` and `value`,
- `formData.append(name, blob, fileName)` – add a field as if it were `<input type="file">`, the third argument `fileName` sets file name (not form field name), as it were a name of the file in user's filesystem,
- `formData.delete(name)` – remove the field with the given `name`,
- `formData.get(name)` – get the value of the field with the given `name`,
- `formData.has(name)` – if there exists a field with the given `name`, returns `true`, otherwise `false`

A form is technically allowed to have many fields with the same `name`, so multiple calls to `append` add more same-named fields.

There's also method `set`, with the same syntax as `append`. The difference is that `.set` removes all fields with the given `name`, and then appends a new field. So it

makes sure there's only one field with such `name`, the rest is just like `append`:

- `formData.set(name, value)`,
- `formData.set(name, blob, fileName)`.

Also we can iterate over `formData` fields using `for..of` loop:

```
let formData = new FormData();
formData.append('key1', 'value1');
formData.append('key2', 'value2');

// List key/value pairs
for(let [name, value] of formData) {
  alert(`${name} = ${value}`); // key1 = value1, then key2 = value2
}
```

Sending a form with a file

The form is always sent as `Content-Type: multipart/form-data`, this encoding allows to send files. So, `<input type="file">` fields are sent also, similar to a usual form submission.

Here's an example with such form:

```
<form id="formElem">
  <input type="text" name="firstName" value="John">
  Picture: <input type="file" name="picture" accept="image/*">
  <input type="submit">
</form>

<script>
  formElem.onsubmit = async (e) => {
    e.preventDefault();

    let response = await fetch('/article/formdata/post/user-avatar', {
      method: 'POST',
      body: new FormData(formElem)
    });

    let result = await response.json();

    alert(result.message);
  };
</script>
```

Picture: No file chosen

Sending a form with Blob data

As we've seen in the chapter [Fetch](#), it's easy to send dynamically generated binary data e.g. an image, as `Blob`. We can supply it directly as `fetch` parameter `body`.

In practice though, it's often convenient to send an image not separately, but as a part of the form, with additional fields, such as "name" and other metadata.

Also, servers are usually more suited to accept multipart-encoded forms, rather than raw binary data.

This example submits an image from `<canvas>`, along with some other fields, as a form, using `FormData`:

```
<body style="margin:0">
  <canvas id="canvasElem" width="100" height="80" style="border:1px solid"></c

  <input type="button" value="Submit" onclick="submit()">

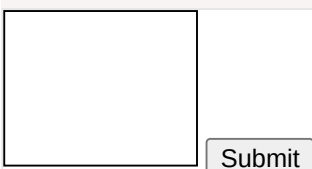
  <script>
    canvasElem.onmousemove = function(e) {
      let ctx = canvasElem.getContext('2d');
      ctx.lineTo(e.clientX, e.clientY);
      ctx.stroke();
    };

    async function submit() {
      let imageBlob = await new Promise(resolve => canvasElem.toBlob(resolve,

      let formData = new FormData();
      formData.append("firstName", "John");
      formData.append("image", imageBlob, "image.png");

      let response = await fetch('/article/formdata/post/image-form', {
        method: 'POST',
        body: formData
      });
      let result = await response.json();
      alert(result.message);
    }

  </script>
</body>
```



Please note how the image `Blob` is added:

```
formData.append("image", imageBlob, "image.png");
```

That's same as if there were `<input type="file" name="image">` in the form, and the visitor submitted a file named `"image.png"` (3rd argument) with the data `imageBlob` (2nd argument) from their filesystem.

The server reads form data and the file, as if it were a regular form submission.

Summary

[FormData](#) objects are used to capture HTML form and submit it using `fetch` or another network method.

We can either create `new FormData(form)` from an HTML form, or create an object without a form at all, and then append fields with methods:

- `formData.append(name, value)`
- `formData.append(name, blob, fileName)`
- `formData.set(name, value)`
- `formData.set(name, blob, fileName)`

Let's note two peculiarities here:

1. The `set` method removes fields with the same name, `append` doesn't. That's the only difference between them.
2. To send a file, 3-argument syntax is needed, the last argument is a file name, that normally is taken from user filesystem for `<input type="file">`.

Other methods are:

- `formData.delete(name)`
- `formData.get(name)`
- `formData.has(name)`

That's it!

Fetch: Download progress

The `fetch` method allows to track *download* progress.

Please note: there's currently no way for `fetch` to track *upload* progress. For that purpose, please use [XMLHttpRequest](#), we'll cover it later.

To track download progress, we can use `response.body` property. It's a `ReadableStream` – a special object that provides body chunk-by-chunk, as it

comes. Readable streams are described in the [Streams API](#) specification.

Unlike `response.text()`, `response.json()` and other methods, `response.body` gives full control over the reading process, and we can count how much is consumed at any moment.

Here's the sketch of code that reads the response from `response.body`:

```
// instead of response.json() and other methods
const reader = response.body.getReader();

// infinite loop while the body is downloading
while(true) {
  // done is true for the last chunk
  // value is Uint8Array of the chunk bytes
  const {done, value} = await reader.read();

  if (done) {
    break;
  }

  console.log(`Received ${value.length} bytes`)
}
```

The result of `await reader.read()` call is an object with two properties:

- **done** – `true` when the reading is complete, otherwise `false`.
- **value** – a typed array of bytes: `Uint8Array`.

i Please note:

Streams API also describes asynchronous iteration over `ReadableStream` with `for await...of` loop, but it's not yet widely supported (see [browser issues](#)), so we use `while` loop.

We receive response chunks in the loop, until the loading finishes, that is: until `done` becomes `true`.

To log the progress, we just need for every received fragment `value` to add its length to the counter.

Here's the full working example that gets the response and logs the progress in console, more explanations to follow:

```
// Step 1: start the fetch and obtain a reader
let response = await fetch('https://api.github.com/repos/javascript-tutorial/e

const reader = response.body.getReader();
```

```

// Step 2: get total length
const contentLength = +response.headers.get('Content-Length');

// Step 3: read the data
let receivedLength = 0; // received that many bytes at the moment
let chunks = []; // array of received binary chunks (comprises the body)
while(true) {
  const {done, value} = await reader.read();

  if (done) {
    break;
  }

  chunks.push(value);
  receivedLength += value.length;

  console.log(`Received ${receivedLength} of ${contentLength}`)
}

// Step 4: concatenate chunks into single Uint8Array
let chunksAll = new Uint8Array(receivedLength); // (4.1)
let position = 0;
for(let chunk of chunks) {
  chunksAll.set(chunk, position); // (4.2)
  position += chunk.length;
}

// Step 5: decode into a string
let result = new TextDecoder("utf-8").decode(chunksAll);

// We're done!
let commits = JSON.parse(result);
alert(commits[0].author.login);

```

Let's explain that step-by-step:

1. We perform `fetch` as usual, but instead of calling `response.json()`, we obtain a stream reader `response.body.getReader()`.

Please note, we can't use both these methods to read the same response: either use a reader or a response method to get the result.

2. Prior to reading, we can figure out the full response length from the `Content-Length` header.

It may be absent for cross-origin requests (see chapter [Fetch: Cross-Origin Requests](#)) and, well, technically a server doesn't have to set it. But usually it's at place.

3. Call `await reader.read()` until it's done.

We gather response chunks in the array `chunks`. That's important, because after the response is consumed, we won't be able to "re-read" it using `response.json()` or another way (you can try, there'll be an error).

4. At the end, we have `chunks` – an array of `Uint8Array` byte chunks. We need to join them into a single result. Unfortunately, there's no single method that concatenates those, so there's some code to do that:

1. We create `chunksAll = new Uint8Array(receivedLength)` – a same-typed array with the combined length.
2. Then use `.set(chunk, position)` method to copy each `chunk` one after another in it.

5. We have the result in `chunksAll`. It's a byte array though, not a string.

To create a string, we need to interpret these bytes. The built-in `TextDecoder` does exactly that. Then we can `JSON.parse` it, if necessary.

What if we need binary content instead of a string? That's even simpler. Replace steps 4 and 5 with a single line that creates a `Blob` from all chunks:

```
let blob = new Blob(chunks);
```

At the end we have the result (as a string or a blob, whatever is convenient), and progress-tracking in the process.

Once again, please note, that's not for *upload* progress (no way now with `fetch`), only for *download* progress.

Also, if the size is unknown, we should check `receivedLength` in the loop and break it once it reaches a certain limit. So that the `chunks` won't overflow the memory.

Fetch: Abort

As we know, `fetch` returns a promise. And JavaScript generally has no concept of "aborting" a promise. So how can we cancel an ongoing `fetch`? E.g. if the user actions on our site indicate that the `fetch` isn't needed any more.

There's a special built-in object for such purposes: `AbortController`. It can be used to abort not only `fetch`, but other asynchronous tasks as well.

The usage is very straightforward:

The AbortController object

Create a controller:

```
let controller = new AbortController();
```

A controller is an extremely simple object.

- It has a single method `abort()`,
- And a single property `signal` that allows to set event listeners on it.

When `abort()` is called:

- `controller.signal` emits the "abort" event.
- `controller.signal.aborted` property becomes `true`.

Generally, we have two parties in the process:

1. The one that performs a cancelable operation, it sets a listener on `controller.signal`.
2. The one that cancels: it calls `controller.abort()` when needed.

Here's the full example (without `fetch` yet):

```
let controller = new AbortController();
let signal = controller.signal;

// The party that performs a cancelable operation
// gets the "signal" object
// and sets the listener to trigger when controller.abort() is called
signal.addEventListener('abort', () => alert("abort!"));

// The other party, that cancels (at any point later):
controller.abort(); // abort!

// The event triggers and signal.aborted becomes true
alert(signal.aborted); // true
```

As we can see, `AbortController` is just a mean to pass `abort` events when `abort()` is called on it.

We could implement the same kind of event listening in our code on our own, without the `AbortController` object.

But what's valuable is that `fetch` knows how to work with the `AbortController` object. It's integrated in it.

Using with fetch

To be able to cancel `fetch`, pass the `signal` property of an `AbortController` as a `fetch` option:

```
let controller = new AbortController();
fetch(url, {
  signal: controller.signal
});
```

The `fetch` method knows how to work with `AbortController`. It will listen to `abort` events on `signal`.

Now, to abort, call `controller.abort()`:

```
controller.abort();
```

We're done: `fetch` gets the event from `signal` and aborts the request.

When a fetch is aborted, its promise rejects with an error `AbortError`, so we should handle it, e.g. in `try..catch`.

Here's the full example with `fetch` aborted after 1 second:

```
// abort in 1 second
let controller = new AbortController();
setTimeout(() => controller.abort(), 1000);

try {
  let response = await fetch('/article/fetch-abort/demo/hang', {
    signal: controller.signal
  });
} catch(err) {
  if (err.name === 'AbortError') { // handle abort()
    alert("Aborted!");
  } else {
    throw err;
  }
}
```

AbortController is scalable

`AbortController` is scalable. It allows to cancel multiple fetches at once.

Here's a sketch of code that fetches many `urls` in parallel, and uses a single controller to abort them all:

```

let urls = [...]; // a list of urls to fetch in parallel

let controller = new AbortController();

// an array of fetch promises
let fetchJobs = urls.map(url => fetch(url, {
  signal: controller.signal
}));

let results = await Promise.all(fetchJobs);

// if controller.abort() is called from anywhere,
// it aborts all fetches

```

If we have our own asynchronous tasks, different from `fetch`, we can use a single `AbortController` to stop those, together with fetches.

We just need to listen to its `abort` event in our tasks:

```

let urls = [...];
let controller = new AbortController();

let ourJob = new Promise((resolve, reject) => { // our task
  ...
  controller.signal.addEventListener('abort', reject);
});

let fetchJobs = urls.map(url => fetch(url, { // fetches
  signal: controller.signal
}));

// Wait for fetches and our task in parallel
let results = await Promise.all([...fetchJobs, ourJob]);

// if controller.abort() is called from anywhere,
// it aborts all fetches and ourJob

```

Summary

- `AbortController` is a simple object that generates an `abort` event on its `signal` property when the `abort()` method is called (and also sets `signal.aborted` to `true`).
- `fetch` integrates with it: we pass the `signal` property as the option, and then `fetch` listens to it, so it's possible to abort the `fetch`.
- We can use `AbortController` in our code. The “call `abort()`” → “listen to `abort` event” interaction is simple and universal. We can use it even without `fetch`.

Fetch: Cross-Origin Requests

If we send a `fetch` request to another web-site, it will probably fail.

For instance, let's try fetching `http://example.com`:

```
try {
  await fetch('http://example.com');
} catch(err) {
  alert(err); // Failed to fetch
}
```

Fetch fails, as expected.

The core concept here is *origin* – a domain/port/protocol triplet.

Cross-origin requests – those sent to another domain (even a subdomain) or protocol or port – require special headers from the remote side.

That policy is called “CORS”: Cross-Origin Resource Sharing.

Why is CORS needed? A brief history

CORS exists to protect the internet from evil hackers.

Seriously. Let's make a very brief historical digression.

For many years a script from one site could not access the content of another site.

That simple, yet powerful rule was a foundation of the internet security. E.g. an evil script from website `hacker.com` could not access the user's mailbox at website `gmail.com`. People felt safe.

JavaScript also did not have any special methods to perform network requests at that time. It was a toy language to decorate a web page.

But web developers demanded more power. A variety of tricks were invented to work around the limitation and make requests to other websites.

Using forms

One way to communicate with another server was to submit a `<form>` there. People submitted it into `<iframe>`, just to stay on the current page, like this:

```
<!-- form target -->
<iframe name="iframe"></iframe>

<!-- a form could be dynamically generated and submitted by JavaScript -->
<form target="iframe" method="POST" action="http://another.com/...">
```

```
...  
</form>
```

So, it was possible to make a GET/POST request to another site, even without networking methods, as forms can send data anywhere. But as it's forbidden to access the content of an `<iframe>` from another site, it wasn't possible to read the response.

To be precise, there were actually tricks for that, they required special scripts at both the iframe and the page. So the communication with the iframe was technically possible. Right now there's no point to go into details, let these dinosaurs rest in peace.

Using scripts

Another trick was to use a `script` tag. A script could have any `src`, with any domain, like `<script src="http://another.com/...">`. It's possible to execute a script from any website.

If a website, e.g. `another.com` intended to expose data for this kind of access, then a so-called "JSONP (JSON with padding)" protocol was used.

Here's how it worked.

Let's say we, at our site, need to get the data from `http://another.com`, such as the weather:

1. First, in advance, we declare a global function to accept the data, e.g. `gotWeather`.

```
// 1. Declare the function to process the weather data  
function gotWeather({ temperature, humidity }) {  
  alert(`temperature: ${temperature}, humidity: ${humidity}`);  
}
```

2. Then we make a `<script>` tag with `src="http://another.com/weather.json?callback=gotWeather"`, using the name of our function as the `callback` URL-parameter.

```
let script = document.createElement('script');  
script.src = `http://another.com/weather.json?callback=gotWeather`;  
document.body.append(script);
```

3. The remote server `another.com` dynamically generates a script that calls `gotWeather(...)` with the data it wants us to receive.

```
// The expected answer from the server looks like this:
gotWeather({
  temperature: 25,
  humidity: 78
});
```

4. When the remote script loads and executes, `gotWeather` runs, and, as it's our function, we have the data.

That works, and doesn't violate security, because both sides agreed to pass the data this way. And, when both sides agree, it's definitely not a hack. There are still services that provide such access, as it works even for very old browsers.

After a while, networking methods appeared in browser JavaScript.

At first, cross-origin requests were forbidden. But as a result of long discussions, cross-origin requests were allowed, but with any new capabilities requiring an explicit allowance by the server, expressed in special headers.

Safe requests

There are two types of cross-origin requests:

1. Safe requests.
2. All the others.

Safe Requests are simpler to make, so let's start with them.

A request is safe if it satisfies two conditions:

1. **Safe method** [↗](#): GET, POST or HEAD
2. **Safe headers** [↗](#) – the only allowed custom headers are:
 - `Accept`,
 - `Accept-Language`,
 - `Content-Language`,
 - `Content-Type` with the value `application/x-www-form-urlencoded`, `multipart/form-data` or `text/plain`.

Any other request is considered "unsafe". For instance, a request with `PUT` method or with an `API-Key` HTTP-header does not fit the limitations.

The essential difference is that a safe request can be made with a `<form>` or a `<script>`, without any special methods.

So, even a very old server should be ready to accept a safe request.

Contrary to that, requests with non-standard headers or e.g. method `DELETE` can't be created this way. For a long time JavaScript was unable to do such requests. So an old server may assume that such requests come from a privileged source, "because a webpage is unable to send them".

When we try to make a unsafe request, the browser sends a special "preflight" request that asks the server – does it agree to accept such cross-origin requests, or not?

And, unless the server explicitly confirms that with headers, an unsafe request is not sent.

Now we'll go into details.

CORS for safe requests

If a request is cross-origin, the browser always adds the `Origin` header to it.

For instance, if we request `https://anywhere.com/request` from `https://javascript.info/page`, the headers will look like:

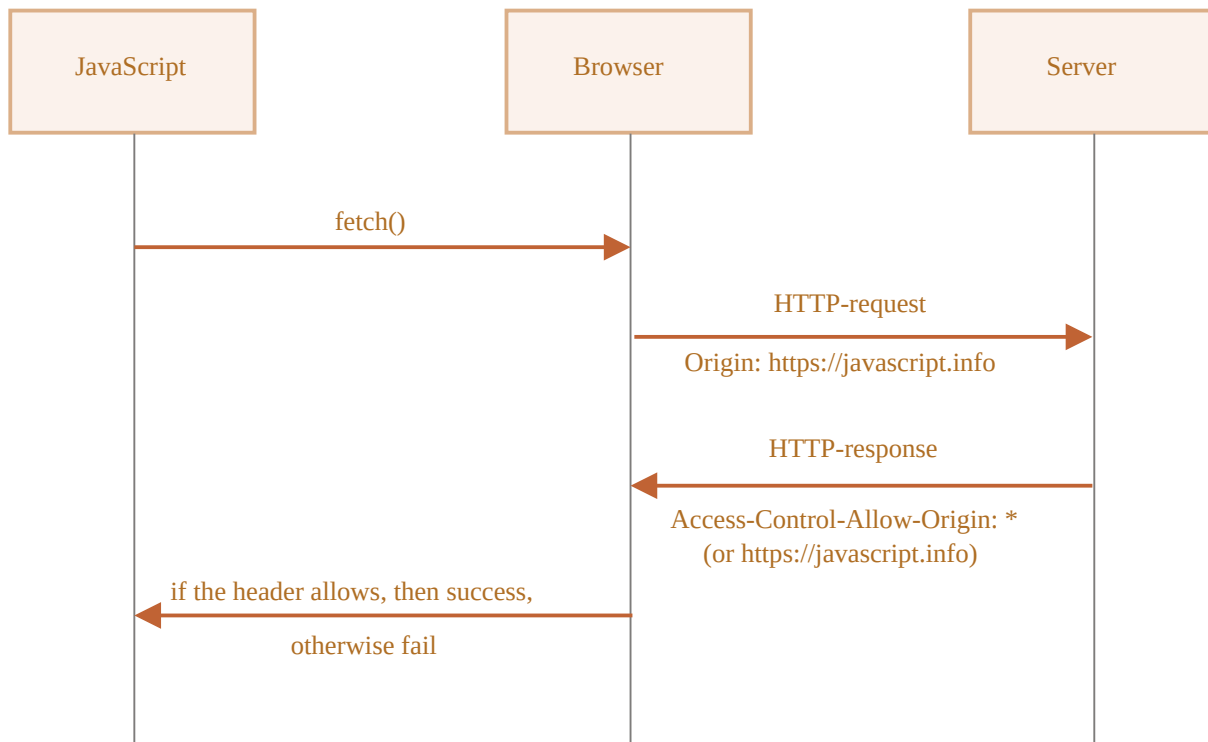
```
GET /request
Host: anywhere.com
Origin: https://javascript.info
...
```

As you can see, the `Origin` header contains exactly the origin (domain/protocol/port), without a path.

The server can inspect the `Origin` and, if it agrees to accept such a request, add a special header `Access-Control-Allow-Origin` to the response. That header should contain the allowed origin (in our case `https://javascript.info`), or a star `*`. Then the response is successful, otherwise it's an error.

The browser plays the role of a trusted mediator here:

1. It ensures that the correct `Origin` is sent with a cross-origin request.
2. It checks for permitting `Access-Control-Allow-Origin` in the response, if it exists, then JavaScript is allowed to access the response, otherwise it fails with an error.



Here's an example of a permissive server response:

```
200 OK
Content-Type:text/html; charset=UTF-8
Access-Control-Allow-Origin: https://javascript.info
```

Response headers

For cross-origin request, by default JavaScript may only access so-called “safe” response headers:

- Cache-Control
- Content-Language
- Content-Length
- Content-Type
- Expires
- Last-Modified
- Pragma

Accessing any other response header causes an error.

To grant JavaScript access to any other response header, the server must send the `Access-Control-Expose-Headers` header. It contains a comma-separated list of unsafe header names that should be made accessible.

For example:

```
200 OK
Content-Type:text/html; charset=UTF-8
Content-Length: 12345
Content-Encoding: gzip
API-Key: 2c9de507f2c54aa1
Access-Control-Allow-Origin: https://javascript.info
Access-Control-Expose-Headers: Content-Encoding,API-Key
```

With such an `Access-Control-Expose-Headers` header, the script is allowed to read the `Content-Encoding` and `API-Key` headers of the response.

“Unsafe” requests

We can use any HTTP-method: not just `GET/POST`, but also `PATCH`, `DELETE` and others.

Some time ago no one could even imagine that a webpage could make such requests. So there may still exist webservices that treat a non-standard method as a signal: “That’s not a browser”. They can take it into account when checking access rights.

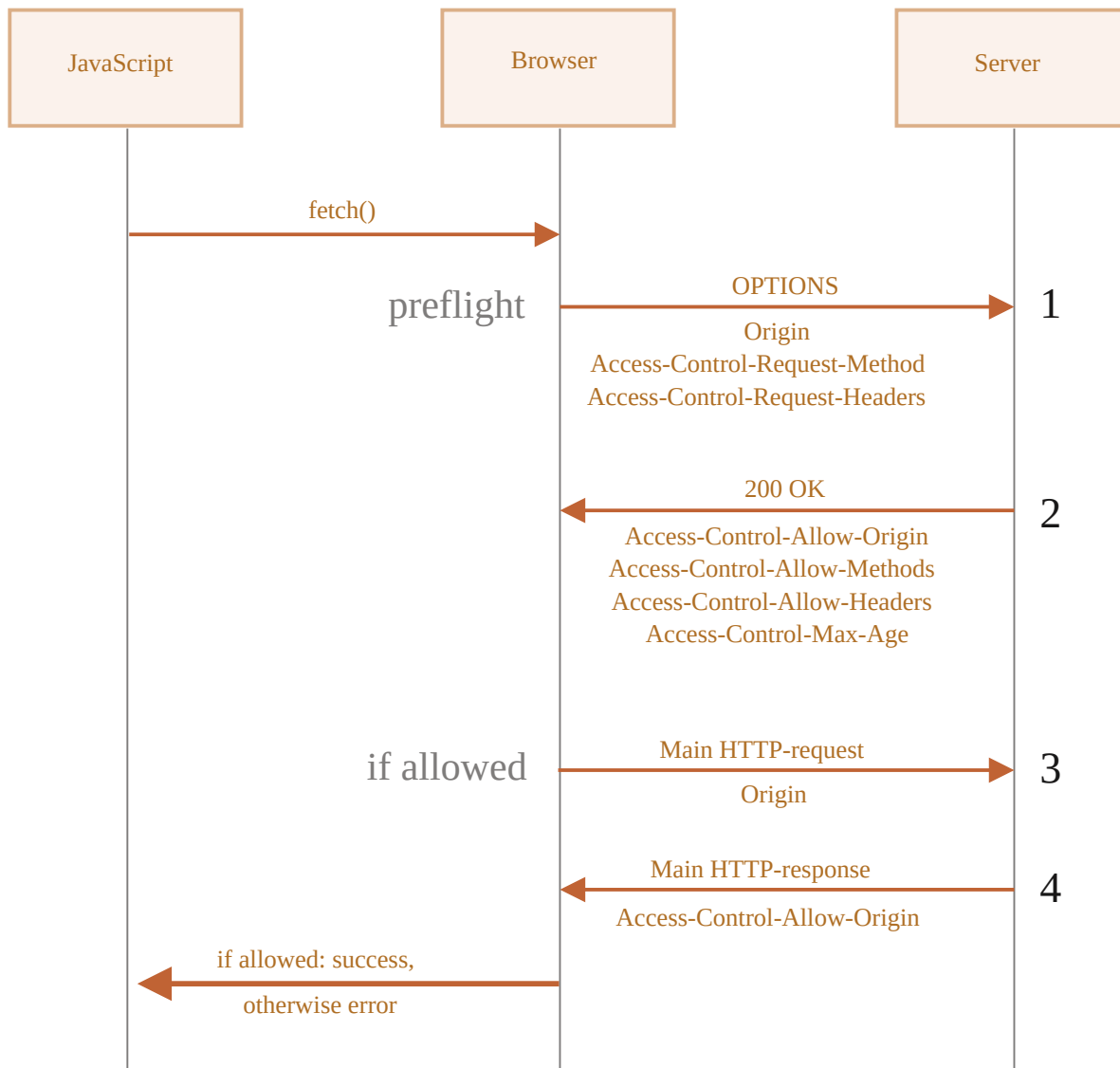
So, to avoid misunderstandings, any “unsafe” request – that couldn’t be done in the old times, the browser does not make such requests right away. First, it sends a preliminary, so-called “preflight” request, to ask for permission.

A preflight request uses the method `OPTIONS`, no body and three headers:

- `Access-Control-Request-Method` header has the method of the unsafe request.
- `Access-Control-Request-Headers` header provides a comma-separated list of its unsafe HTTP-headers.
- `Origin` header tells from where the request came. (such as `https://javascript.info`)

If the server agrees to serve the requests, then it should respond with empty body, status 200 and headers:

- `Access-Control-Allow-Origin` must be either `*` or the requesting origin, such as `https://javascript.info`, to allow it.
- `Access-Control-Allow-Methods` must have the allowed method.
- `Access-Control-Allow-Headers` must have a list of allowed headers.
- Additionally, the header `Access-Control-Max-Age` may specify a number of seconds to cache the permissions. So the browser won’t have to send a preflight for subsequent requests that satisfy given permissions.



Let's see how it works step-by-step on the example of a cross-origin `PATCH` request (this method is often used to update data):

```

let response = await fetch('https://site.com/service.json', {
  method: 'PATCH',
  headers: {
    'Content-Type': 'application/json',
    'API-Key': 'secret'
  }
});

```

There are three reasons why the request is unsafe (one is enough):

- Method `PATCH`
- `Content-Type` is not one of: `application/x-www-form-urlencoded`, `multipart/form-data`, `text/plain`.
- “Unsafe” `API-Key` header.

Step 1 (preflight request)

Prior to sending such a request, the browser, on its own, sends a preflight request that looks like this:

```
OPTIONS /service.json
Host: site.com
Origin: https://javascript.info
Access-Control-Request-Method: PATCH
Access-Control-Request-Headers: Content-Type,API-Key
```

- Method: `OPTIONS`.
- The path – exactly the same as the main request: `/service.json`.
- Cross-origin special headers:
 - `Origin` – the source origin.
 - `Access-Control-Request-Method` – requested method.
 - `Access-Control-Request-Headers` – a comma-separated list of “unsafe” headers.

Step 2 (preflight response)

The server should respond with status 200 and the headers:

- `Access-Control-Allow-Origin: https://javascript.info`
- `Access-Control-Allow-Methods: PATCH`
- `Access-Control-Allow-Headers: Content-Type,API-Key`.

That allows future communication, otherwise an error is triggered.

If the server expects other methods and headers in the future, it makes sense to allow them in advance by adding them to the list.

For example, this response also allows `PUT`, `DELETE` and additional headers:

```
200 OK
Access-Control-Allow-Origin: https://javascript.info
Access-Control-Allow-Methods: PUT,PATCH,DELETE
Access-Control-Allow-Headers: API-Key,Content-Type,If-Modified-Since,Cache-Con
Access-Control-Max-Age: 86400
```

Now the browser can see that `PATCH` is in `Access-Control-Allow-Methods` and `Content-Type,API-Key` are in the list `Access-Control-Allow-Headers`, so it sends out the main request.

If there's the header `Access-Control-Max-Age` with a number of seconds, then the preflight permissions are cached for the given time. The response above will be

cached for 86400 seconds (one day). Within this timeframe, subsequent requests will not cause a preflight. Assuming that they fit the cached allowances, they will be sent directly.

Step 3 (actual request)

When the preflight is successful, the browser now makes the main request. The process here is the same as for safe requests.

The main request has the `Origin` header (because it's cross-origin):

```
PATCH /service.json
Host: site.com
Content-Type: application/json
API-Key: secret
Origin: https://javascript.info
```

Step 4 (actual response)

The server should not forget to add `Access-Control-Allow-Origin` to the main response. A successful preflight does not relieve from that:

```
Access-Control-Allow-Origin: https://javascript.info
```

Then JavaScript is able to read the main server response.

Please note:

Preflight request occurs “behind the scenes”, it’s invisible to JavaScript.

JavaScript only gets the response to the main request or an error if there’s no server permission.

Credentials

A cross-origin request initiated by JavaScript code by default does not bring any credentials (cookies or HTTP authentication).

That’s uncommon for HTTP-requests. Usually, a request to `http://site.com` is accompanied by all cookies from that domain. Cross-origin requests made by JavaScript methods on the other hand are an exception.

For example, `fetch('http://another.com')` does not send any cookies, even those (!) that belong to `another.com` domain.

Why?

That's because a request with credentials is much more powerful than without them. If allowed, it grants JavaScript the full power to act on behalf of the user and access sensitive information using their credentials.

Does the server really trust the script that much? Then it must explicitly allow requests with credentials with an additional header.

To send credentials in `fetch`, we need to add the option `credentials: "include"`, like this:

```
fetch('http://another.com', {  
  credentials: "include"  
});
```

Now `fetch` sends cookies originating from `another.com` with request to that site.

If the server agrees to accept the request *with credentials*, it should add a header `Access-Control-Allow-Credentials: true` to the response, in addition to `Access-Control-Allow-Origin`.

For example:

```
200 OK  
Access-Control-Allow-Origin: https://javascript.info  
Access-Control-Allow-Credentials: true
```

Please note: `Access-Control-Allow-Origin` is prohibited from using a star `*` for requests with credentials. Like shown above, it must provide the exact origin there. That's an additional safety measure, to ensure that the server really knows who it trusts to make such requests.

Summary

From the browser point of view, there are two kinds of cross-origin requests: “safe” and all the others.

“Safe” requests must satisfy the following conditions:

- Method: GET, POST or HEAD.
- Headers – we can set only:
 - `Accept`
 - `Accept-Language`
 - `Content-Language`

- `Content-Type` to the value `application/x-www-form-urlencoded`, `multipart/form-data` or `text/plain`.

The essential difference is that safe requests were doable since ancient times using `<form>` or `<script>` tags, while unsafe were impossible for browsers for a long time.

So, the practical difference is that safe requests are sent right away, with the `Origin` header, while for the other ones the browser makes a preliminary “preflight” request, asking for permission.

For safe requests:

- → The browser sends the `Origin` header with the origin.
- ← For requests without credentials (not sent by default), the server should set:
 - `Access-Control-Allow-Origin` to `*` or same value as `Origin`
- ← For requests with credentials, the server should set:
 - `Access-Control-Allow-Origin` to same value as `Origin`
 - `Access-Control-Allow-Credentials` to `true`

Additionally, to grant JavaScript access to any response headers except `Cache-Control`, `Content-Language`, `Content-Type`, `Expires`, `Last-Modified` or `Pragma`, the server should list the allowed ones in `Access-Control-Expose-Headers` header.

For unsafe requests, a preliminary “preflight” request is issued before the requested one:

- → The browser sends an `OPTIONS` request to the same URL, with the headers:
 - `Access-Control-Request-Method` has requested method.
 - `Access-Control-Request-Headers` lists unsafe requested headers.
- ← The server should respond with status 200 and the headers:
 - `Access-Control-Allow-Methods` with a list of allowed methods,
 - `Access-Control-Allow-Headers` with a list of allowed headers,
 - `Access-Control-Max-Age` with a number of seconds to cache the permissions.
- Then the actual request is sent, and the previous “safe” scheme is applied.

✓ Tasks

Why do we need Origin?

importance: 5